Documentación técnica del producto
Prototipo BI asistido por inteligencia artificial

Manual técnico

Instalación, operación y réplica del entorno

Prueba de concepto de un prototipo funcional de BI asistido por IA para la construcción semi-automatizada y supervisada de un datamart de ventas

Responsables	Emily Dayan Robles Alvarado y Lesly Samay Velásquez Anchundia

Código documental	MAN-TEC-01
Versión del manual	5.2.1
Línea base	Ramas `develop` y `main`
Repositorio	LFXAS/prototipo-bi-ia
Clasificación	Documentación técnica - repositorio público
Fecha	5 de noviembre de 2026

Índice

1. Propósito y audiencia **1**

2. Arquitectura operativa **1**

2.1. Servicios . . . 2

2.2. Estructura del repositorio . . . 2

3. Requisitos previos **2**

3.1. Hardware recomendado . . . 2

3.2. Software requerido . . . 3

4. Configuración segura **3**

4.1. Archivo de entorno . . . 3

5. Instalación de desarrollo en este computador **4**

6. Instalación en otro computador desde el código **5**

6.1. Obtener el repositorio . . . 5

6.2. Preparar secretos . . . 5

6.3. Construir y levantar . . . 5

6.4. Confirmar el estado . . . 5

6.5. Abrir la aplicación . . . 5

7. Vista local de entrega sin reemplazar desarrollo **6**

8. Operación funcional de los Sprints 2 a 5 **7**

8.1. Migraciones y datos semilla por equipo . . . 7

8.2. Acceso inicial y sesión . . . 7

8.3. Navegación administrativa . . . 7

8.4. Usuarios, roles, permisos y menús . . . 8

8.5. Eliminación y auditoría . . . 8

8.6. Parámetros generales . . . 9

8.7. Conexiones de datos . . . 9

8.8. Instantáneas y explorador de esquema . . . 10

8.9. Restauración académica de WideWorldImporters . . . 10

8.10. Asistente de datamart supervisado . . . 10

8.11. Sugerir y comprobar una necesidad desde los metadatos . . . 12

8.12. Corrección posterior al cierre: revisión del 4–5 de noviembre de 2026 . . . 13
8.13. Validación acumulativa visible . . . 16
8.14. Materialización controlada del Sprint 4 . . . 17
8.15. Copiloto contextual para decisiones semánticas . . . 18
8.16. Resolución descriptiva y ejecución corregida del Sprint 5 . . . 18
8.17. Analítica explicable y conversación contextual . . . 19
8.18. Comprobar promedios y evento comercial . . . 20
8.19. Recuperación después de vencer la sesión . . . 20
8.20. Exportación PDF y Excel . . . 21
8.21. Configuración LLM cloud y local . . . 22
8.22. Selección operativa de Groq o Claude . . . 23
8.23. Alternativa LLM local con Ollama . . . 23
8.24. Preparar y descargar el modelo . . . 24
8.25. Registrar y probar la configuración . . . 24
8.26. Detener sin borrar el modelo . . . 24

9. Ejecución desde imágenes de GHCR 25
9.1. Autenticarse en GHCR . . . 25
9.2. Configurar la entrega . . . 25
9.3. Descargar e iniciar . . . 25

10. Comprobación técnica 25
10.1. Comprobación rápida . . . 25
10.2. Comprobación completa . . . 26
10.3. Contrato OpenAPI . . . 26
10.4. Validación de AdventureWorks . . . 27

11. Operación completa de los contenedores 27
11.1. Diferencia entre pausar, reanudar y retirar . . . 27
11.2. Ciclo del ambiente de desarrollo . . . 27
11.3. Ciclo del LLM local opcional . . . 28
11.4. Ciclo de la vista local de entrega . . . 28
11.5. Ciclo de la entrega completa desde GHCR . . . 29
11.6. Detener todos los ambientes conservando datos . . . 29
11.7. Referencia de comandos abreviados . . . 30

12. Persistencia y reinicio de datos **30**
12.1. Detener sin borrar . . . 30
12.2. Reconstruir desde cero . . . 30
12.3. Qué se versiona . . . 31

13. Actualización y reversión **31**
13.1. Entorno construido desde código . . . 31
13.2. Entorno basado en GHCR . . . 31

14. Flujo Git y CI/CD **31**
14.1. Desarrollo por rama . . . 31
14.2. Integración continua . . . 32
14.3. Entrega continua . . . 32
14.4. Cierre propuesto en Azure . . . 32
14.5. Cómo verificar CI y CD en GitHub . . . 33
14.6. Qué significa DevOps en este prototipo . . . 35

15. Desarrollo guiado por especificaciones (SDD) **36**
15.1. Procedimiento SDD para cada función . . . 36

16. Construcción operativa del Sprint 1 desde cero **37**
16.1. Secuencia de construcción . . . 37
16.2. Primera instalación manual reproducible . . . 37

17. Referencia comentada de configuración **38**
17.1. Archivos de entorno . . . 38
17.2. Compose de desarrollo: `compose.yaml` . . . 38
17.3. Compose de producción local: `compose.prod.yaml` . . . 39
17.4. Compose de entrega: `compose.release.yaml` . . . 39
17.5. Dockerfile del backend . . . 40
17.6. Dockerfile del frontend . . . 40
17.7. Imágenes de infraestructura . . . 40

18. Automatización local con Make y scripts **40**

19. Git y GitHub: procedimiento completo **41**
19.1. Inicialización conceptual del repositorio . . . 41
19.2. Trabajo diario y revisión . . . 41

19.3. Protecciones que deben configurarse en GitHub 42

20. Integración continua: lectura de `ci.yml` **42**

20.1. Disparadores, concurrencia y permisos 42

20.2. Trabajos y dependencia entre resultados 42

21. Entrega continua: lectura de `cd.yml` **43**

21.1. Qué hace y qué no hace el CD actual 43

22. Dependabot y mantenimiento controlado **43**

23. Persistencia, datos y paso a producción **43**

24. Prueba final y evidencia del Sprint 1 **44**

24.1. Errores de escritura frecuentes 44

25. Seguridad operativa **44**

26. Diagnóstico de fallos **44**

26.1. Registros útiles 45

27. Lista de comprobación para una demostración **45**

28. Mantenimiento de la documentación **46**

29. Responsabilidades y escalamiento **46**

A. Referencia rápida de URLs y puertos **47**

B. Recuperación mínima **48**

C. Control del manual **48**

1. Propósito y audiencia

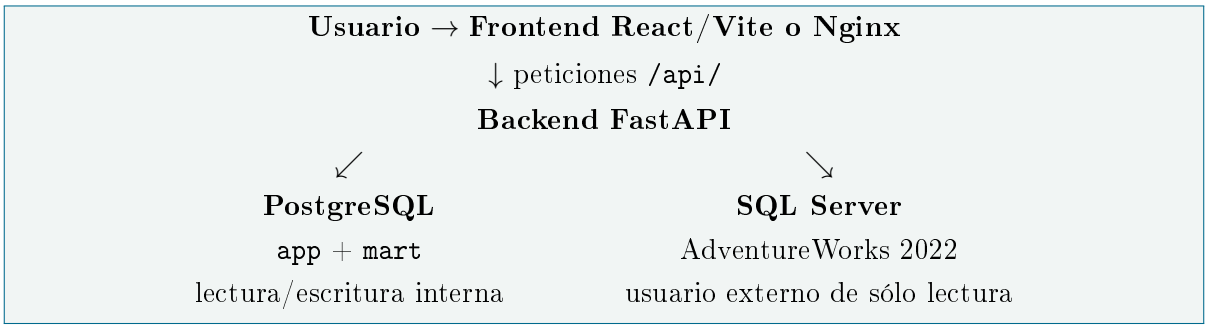
Este manual permite instalar, levantar, comprobar, mantener y replicar el ambiente técnico del prototipo. Está dirigido a las autoras, docentes revisores, colaboradores autorizados y personal técnico que necesite reproducir la demostración.

El documento cubre tres modalidades:

1. **Desarrollo desde el código**: clona el repositorio y construye las imágenes localmente.
2. **Vista local de entrega**: añade Nginx en el puerto 8080 y reutiliza la API y las bases de desarrollo.
3. **Ejecución de una entrega completa**: descarga imágenes de GitHub Container Registry (GHCR) en un proyecto Compose aislado.

La versión actual conserva la infraestructura del Sprint 1 y el núcleo administrativo del Sprint 2. Sprint 3 añadió conexión SQL Server, introspección y asistente supervisado; Sprint 4 incorporó materialización y conciliación; Sprint 5 completó calidad semántica, analítica y reportería. Sprint 6 permite varias fuentes SQL Server habilitadas, aísla catálogo y expedientes por conexión y valida el mismo ciclo con AdventureWorks2022 y WideWorldImporters.

2. Arquitectura operativa



2.1. Servicios

Servicio	Responsabilidad	Puerto desarrollo	Persistencia
frontend	Interfaz React y proxy a la API	5173	Código y dependencias
frontend-delivery	Frontend compilado servido por Nginx	8080	Sin volumen
backend	API FastAPI, configuración y salud	8000	Código montado
postgres	Configuración interna y datamart materializado	55432	Volumen `postgres_data`
sqlserver	Fuente AdventureWorks externa	51433	Volumen `sqlserver_data`
ollama	LLM local opcional, sólo red interna	No publicado	Volumen `ollama_models`
raíz criptográfica	Clave local para cifrar secretos autorizados	No aplica	Volumen `secret_key_data`

Compose crea la red y los volúmenes de desarrollo bajo el nombre lógico `bi-ia-prototype`. El perfil `delivery` sólo añade Nginx a esa red; el perfil opcional `local-llm` añade Ollama sin publicar el puerto 11434 en el host. El volumen `secret_key_data` se monta exclusivamente en FastAPI. El Compose completo de entrega usa `bi-ia-prototype-release`, otra red, otros puertos y otros volúmenes. Ninguno usa contenedores PostgreSQL o SQL Server ajenos al proyecto.

2.2. Estructura del repositorio

```
backend/                 FastAPI, pruebas y migraciones
frontend/                React, Vite, pruebas y Nginx
infra/postgres/          imagen e inicializacion PostgreSQL
infra/sqlserver/         imagen y restauracion AdventureWorks
infra/docs/              imagen reproducible de LaTeX
scripts/                 arranque, espera y verificacion
docs/                    arquitectura, guias y evidencias
.github/workflows/       integracion y entrega continuas
compose.yaml             entorno de desarrollo autocontenido
compose.release.yaml     entorno con imagenes de GHCR
Makefile                 comandos operativos abreviados
```

3. Requisitos previos

3.1. Hardware recomendado

- Procesador de 64 bits con virtualización habilitada.
- 8 GB de memoria disponibles para Docker; 6 GB es el mínimo práctico.
- 15 GB libres para imágenes, capas, respaldo y volúmenes; reservar al menos 8 GB adicionales si se descargará el modelo local de Ollama.
- Conexión a Internet durante la primera construcción y restauración.

3.2. Software requerido

- Docker Engine 24 o superior y Docker Compose v2; Docker Desktop es válido.
- Git para clonar y actualizar el repositorio.
- Acceso al repositorio público; permiso de escritura sólo para colaboradores autorizados.
- Para consumir imágenes públicas no se necesita iniciar sesión; si un paquete GHCR conserva visibilidad privada, se requieren credenciales con `read:packages`.

No se requiere instalar Python, Node.js, PostgreSQL, SQL Server, controladores ODBC ni LaTeX en el host.

En equipos ARM, incluido Apple Silicon, SQL Server usa `linux/amd64` mediante emulación. El primer arranque puede ser más lento.

4. Configuración segura

4.1. Archivo de entorno

El archivo `.env` contiene la configuración local y nunca debe enviarse a Git. Se crea desde el ejemplo:

```
cp .env.example .env
```

Se deben reemplazar todos los valores que empiecen con `ChangeMe_*`. Las claves deben cumplir la política de SQL Server: longitud suficiente y combinación de mayúsculas, minúsculas, números y símbolos.

Variable	Valor inicial	Uso
`COMPOSE_PROJECT_NAME`	`bi-ia-prototype`	Prefijo aislado de red, contenedores y volúmenes.
`FRONTEND_PORT`	5173	Puerto web de desarrollo; 8080 en la entrega completa.
`DELIVERY_FRONTEND_PORT`	8080	Vista Nginx que convive con Vite.
`BACKEND_PORT`	8000	Puerto de FastAPI y OpenAPI.
`LOCAL_POSTGRES_PORT`	55432	Publicación local de PostgreSQL.
`LOCAL_SQLSERVER_PORT`	51433	Publicación local de SQL Server.
`POSTGRES_DB`	`bi_prototype`	Base interna y datamart del prototipo.
`POSTGRES_USER`	`bi_app`	Cuenta de la aplicación interna.
`POSTGRES_PASSWORD`	secreto local	Clave de PostgreSQL.
`SQLSERVER_DATABASE`	`AdventureWorks2022`	Base externa restaurada.
`SQLSERVER_USER`	`bi_reader`	Cuenta exclusiva de lectura.
`SQLSERVER_PASSWORD`	secreto local	Clave del lector.
`SQLSERVER_SA_PASSWORD`	secreto local	Clave administrativa usada sólo durante inicialización.
`IMAGE_PREFIX`	ruta GHCR	Prefijo de las imágenes publicadas.

Variable	Valor inicial	Uso
`IMAGE_TAG`	`main`	Entrega a ejecutar; se recomienda una etiqueta versionada.
Credencial Gemini/Groq/Qwen	interfaz web	Se registra o reemplaza desde Configuración LLM; no pertenece a `.env`.
`OLLAMA_MODEL`	`qwen2.5:3b`	Modelo local ágil que se descarga explícitamente por equipo.
`OLLAMA_CONTEXT_LENGTH`	4096	Contexto local que permite completar los contratos JSON del asistente sin abandonar un consumo moderado.

Precaución. No copiar contraseñas, tokens, archivos `.env`, respaldos `.bak` ni volúmenes al repositorio. Ante una exposición, rotar inmediatamente la credencial y revisar el historial.

5. Instalación de desarrollo en este computador

El proyecto se encuentra en `/home/ceo/Proyectos/BI`. Para una primera ejecución:

```
cd "/home/ceo/Proyectos/BI"
cp .env.example .env
# Editar .env y cambiar todos los secretos ChangeMe_*
make bootstrap
```

`make bootstrap` realiza las siguientes acciones:

1. valida o crea el archivo de configuración local;
2. construye las imágenes propias;
3. crea una red y tres volúmenes exclusivos;
4. inicia PostgreSQL y espera su estado saludable;
5. inicia SQL Server, descarga y restaura AdventureWorks si el volumen está vacío;
6. crea y prueba el usuario `bi_reader`;
7. aplica las migraciones Alembic pendientes;
8. ejecuta el servicio `seed`, que incorpora sin duplicar el catálogo protegido aprobado;
9. inicia FastAPI y después React;
10. espera hasta que los cuatro servicios respondan correctamente.

El primer arranque puede tardar varios minutos. Para observar la restauración:

```
docker compose logs -f sqlserver
```

6. Instalación en otro computador desde el código

6.1. Obtener el repositorio

El código puede clonarse sin invitación porque el repositorio es público. Para crear ramas en el repositorio central, revisar o fusionar cambios se requiere ser colaborador autorizado.

```
git clone https://github.com/LFXAS/prototipo-bi-ia.git
cd prototipo-bi-ia
```

La carpeta puede ubicarse en cualquier ruta; Compose utiliza rutas relativas.

6.2. Preparar secretos

```
cp .env.example .env
# Abrir .env y reemplazar todos los valores ChangeMe_*
```

Si alguno de los puertos está ocupado, cambiarlo en `.env` antes del arranque.

6.3. Construir y levantar

```
make bootstrap
```

6.4. Confirmar el estado

```
docker compose ps
curl http://localhost:8000/api/v1/health/live
curl http://localhost:8000/api/v1/health/ready
```

Las dos respuestas deben indicar `"status":.°k"`; la segunda también debe indicar `"postgres":.°k"`.

6.5. Abrir la aplicación

- Frontend: http://localhost:5173
- API: http://localhost:8000
- OpenAPI: http://localhost:8000/docs



Figura 1: Resultado esperado después de levantar el entorno de desarrollo.

7. Vista local de entrega sin reemplazar desarrollo

Con los servicios de desarrollo activos, ejecutar:

```
make delivery-preview-up
```

El comando construye la etapa `production` del frontend y añade `frontend-delivery` mediante el perfil Compose `delivery`. No detiene Vite, no recrea el backend y no duplica PostgreSQL ni SQL Server.

- Vite y recarga automática: `http://localhost:5173`
- Frontend compilado con Nginx: `http://localhost:8080`
- API compartida: `http://localhost:8000`

Esta modalidad valida el frontend que se empaquetará, pero no representa un ambiente productivo totalmente independiente porque comparte el backend de desarrollo.

8. Operación funcional de los Sprints 2 a 5

El Sprint 2 se opera sobre el mismo ambiente levantado con `make bootstrap` o `make up`. No se crea una instalación ni un conjunto de contenedores adicional para seguridad: React, FastAPI y PostgreSQL ya contienen el incremento. La migración se aplica mediante el servicio `migrate`; después, el servicio `seed` carga de forma idempotente el catálogo protegido antes de iniciar FastAPI.

8.1. Migraciones y datos semilla por equipo

Cada volumen PostgreSQL pertenece exclusivamente al equipo que lo creó y nunca se transmite por GitHub. Cuando una programadora actualiza `develop` y levanta Compose, Alembic aplica el cambio de estructura y `seed` incorpora o actualiza los permisos, menús, rol administrador y cuenta inicial aprobados. Este orden permite que una base vacía quede operativa y que una base ya existente conserve sus datos locales no protegidos.

Para repetir sólo el catálogo base, sin borrar ni exportar PostgreSQL, se ejecuta desde la raíz del repositorio:

```
make seed
```

La operación no duplica registros. No distribuye cuentas temporales, auditoría, contraseñas, tokens, configuraciones LLM ni datos de negocio. Una semilla demostrativa futura deberá ser sintética, opcional y aprobada con la especificación del módulo que la consuma.

8.2. Acceso inicial y sesión

El primer arranque crea la cuenta administradora inicial desde las variables de administración definidas en `.env`. Estos valores son secretos locales: se cambian antes de una demostración compartida y nunca se incorporan a Git, capturas ni bitácora. Se abre la interfaz en el puerto configurado, se ingresa con esas credenciales y se cierra la sesión desde el encabezado al terminar.

FastAPI valida la contraseña con Argon2 y entrega un JWT de vida limitada. React conserva únicamente el token de sesión necesario para consultar la API; el token no contiene contraseñas, claves LLM ni permisos que sustituyan la comprobación del servidor. Cada endpoint administrativo comprueba de nuevo el usuario activo y su permiso efectivo.

8.3. Navegación administrativa

La navegación muestra sólo menús autorizados y los organiza así:

Grupo o acceso	Pantallas y finalidad
Inicio	Resumen inicial de la sesión y punto de entrada directo.
Preparación del entorno	Conexiones de datos, Explorador de esquema, Configuración LLM y Parámetros.
Diseño y transformación BI	Catálogo analítico, Asistente de datamart y Datamart de ventas.
Análisis y decisiones	Analítica de ventas, copiloto y reportes autorizados.
Administración y seguridad	Usuarios, Roles, Permisos, Menús y Auditoría.

Los grupos se ordenan por el flujo de trabajo y se pueden plegar o desplegar, incluso si contienen la pantalla activa. Cada categoría y subopción incorpora un icono SVG junto a su texto, foco visible y estado activo por fondo y borde; la iconografía nunca sustituye el nombre accesible. En escritorio, el panel lateral se contrae o restaura mediante el chevrón de su borde; en tableta y móvil se abre con menú hamburguesa. Esta reclasificación ocurre sobre las opciones que FastAPI ya autorizó: una URL o petición directa sin permiso continúa recibiendo acceso denegado.

8.4. Usuarios, roles, permisos y menús

La administración no solicita IDs ni códigos técnicos a la persona usuaria. En **Usuarios**, se registra correo, nombre, contraseña inicial y roles mediante una tabla de casillas con nombre y descripción. Durante una edición, dejar la contraseña vacía mantiene la contraseña existente; escribir una nueva la reemplaza de forma segura.

El correo de una cuenta existente se presenta como sólo lectura, porque su endpoint de actualización no admite cambiarlo. Al editar una cuenta protegida, la interfaz omite los roles del envío y conserva únicamente los campos permitidos. Si no puede cargar roles o permisos, bloquea el formulario hasta una recarga satisfactoria; un fallo de consulta no equivale a un catálogo vacío que se pueda guardar.

En **Roles**, los permisos se asignan o retiran mediante otra tabla de casillas legible. La actualización de asociaciones usa el contrato `PUT /roles/{id}/permissions`; el código técnico interno del rol no se digita desde la pantalla. Permisos y Menús son catálogos controlados: una pantalla administrativa no puede inventar un permiso, ruta o módulo que no exista en una especificación, migración y endpoint reales.

Las listas administrativas son paginadas desde el servidor. Usuarios, Roles y Menús permiten crear, editar, activar/desactivar y, cuando corresponde, eliminar definitivamente. Las acciones de registros base de recuperación muestran el estado **Protegido**; FastAPI impide desactivar, eliminar o debilitar la cuenta y rol administrativos iniciales.

8.5. Eliminación y auditoría

Eliminar no significa ocultar un registro. Para una eliminación física permitida, la interfaz pide confirmación y FastAPI comprueba primero las asociaciones hijas. Por ejemplo, un rol con usuarios o permisos asignados no se borra hasta resolver esas relaciones; la API responde con conflicto y una explicación. No existe borrado en cascada silencioso.

Una cuenta temporal no protegida sí puede eliminarse después de retirar sus roles. Sus eventos de auditoría no son una dependencia bloqueante: la migración `20260911_05` configura la referencia de

actor con `ON DELETE SET NULL`. Al borrar la cuenta, el identificador técnico queda vacío, pero cada evento conserva la etiqueta histórica segura del actor (nombre y correo en el momento de la acción). Así se mantiene la trazabilidad sin conservar una cuenta de prueba ni borrar eventos de auditoría.

Toda mutación administrativa genera un evento de auditoría con actor, fecha, acción, tipo de recurso, identificador y resumen seguro. Se registran inicio de sesión, altas, actualizaciones, cambios de estado, asociaciones, pruebas de conexión y eliminaciones. La vista Auditoría es sólo de consulta, paginada e inmutable; para actores ya eliminados muestra la etiqueta histórica y nunca presenta contraseñas, JWT, API keys ni cadenas de conexión completas.

8.6. Parámetros generales

La pantalla Parámetros no es una tabla libre de clave/valor. Sólo muestra cuatro enteros operativos aprobados: `UI_PAGE_SIZE`, `CONNECTION_TIMEOUT_SECONDS`, `LLM_TIMEOUT_SECONDS` y `METADATA_BLOCK_MAX_ITEMS`. El catálogo analítico se administra en **IA >Catálogo analítico** con los permisos independientes `copilot.catalog.read` y `copilot.catalog.write`. Allí se gestionan preguntas de negocio y periodicidades por dominio y por fuente. El objetivo pertenece a cada análisis y las dimensiones no son un parámetro. FastAPI valida y audita cada cambio.

El catálogo tiene dos planos visibles. El **catálogo de negocio** pertenece al analista: define qué preguntas quiere ofrecer y con qué periodicidades. La **cobertura técnica** pertenece a la plataforma: se recalcula desde la instantánea seleccionada y muestra las tablas o columnas reales que respaldan cada capacidad. El LLM interpreta ambos planos y redacta una propuesta, pero no crea evidencia ni determina por sí solo que una capacidad existe. Después de cada respuesta, FastAPI vuelve a validar referencias, tipos, PK, FK, rutas, granularidad, fórmulas y cobertura.

El perfil de ventas aporta cuatro preguntas estándar: evolución temporal, producto, cliente y territorio. En la fuente AdventureWorks el analista añadió manualmente **Costos y rentabilidad de las ventas** y **Ventas y costos por unidad** después de recibir una recomendación conversacional. Estas entradas personalizadas pertenecen a esa configuración y no se copian a otra conexión. Una nueva fuente recibe la línea base; el analista decide si añade preguntas y la plataforma comprueba si sus metadatos permiten resolverlas.

Los tiempos máximos de conexión y LLM controlan sus respectivas operaciones. `LLM_TIMEOUT_SECONDS` inicia en 600 segundos y admite hasta 900 para modelos locales ejecutados por CPU; los proveedores cloud normalmente terminan antes. El tamaño de bloque participa en la preparación del contexto y el perfil de dominio aplica además un alcance técnico acotado para no enviar las 71 tablas completas a un modelo pequeño.

8.7. Conexiones de datos

La pantalla **Conexiones de datos** permite registrar nombre, servidor, puerto, base, usuario, contraseña y opciones controladas de cifrado para SQL Server. No acepta una cadena de conexión libre. La contraseña se cifra en `app.secrets`; React recibe sólo los campos no secretos y nunca recupera el valor guardado. Al editar, dejarla vacía conserva la credencial existente y cualquier cambio invalida la prueba anterior.

Probar conexión abre SQL Server con intención de lectura, consulta el catálogo y comprueba que la cuenta no pertenezca a `db_owner` o `db_datawriter` ni posea permisos de inserción, actualización, eliminación, creación de tablas o control. Una prueba fallida no puede habilitarse. Varias conexiones probadas pueden permanecer habilitadas al mismo tiempo; el selector del espacio de trabajo establece el

contexto de la sesión y no altera el de otro usuario. FastAPI audita creación, edición, prueba, habilitación, deshabilitación y eliminación sin guardar cadenas completas.

En el entorno Docker académico se registra el host `sqlserver`, puerto 1433 y un usuario lector aprovisionado durante el arranque. AdventureWorks2022 y WideWorldImporters pueden coexistir en la misma instancia, pero cada una se registra como conexión independiente. Las variables de `.env` siguen siendo necesarias para construir la instancia local, pero la operación del flujo BI usa la credencial registrada desde la plataforma.

8.8. Instantáneas y explorador de esquema

La acción **Actualizar metadatos** está disponible sobre la fuente seleccionada para personas con el permiso `metadata.refresh`. FastAPI descifra la contraseña sólo en memoria, consulta exclusivamente los catálogos de SQL Server y normaliza esquemas, tablas, columnas, tipos, nulabilidad, claves primarias y claves foráneas. No consulta filas, valores de ventas, procedimientos ni código SQL almacenado.

El documento canónico se ordena de forma estable y se almacena como JSONB en `app.metadata_snapshots`. Su hash SHA-256 permite reconocer la misma estructura: si no cambió, se reutiliza la instantánea vigente y no se duplica. Cada captura conserva conexión, base, versión contractual, totales, actor histórico y fecha. La migración `20260918_08` impide eliminar una conexión que tenga instantáneas asociadas.

La pantalla **Datos >Explorador de esquema** consulta la captura persistida, no la fuente original. Permite buscar por esquema, tabla o columna y presenta detalle de columnas, PK y relaciones entrantes y salientes. La auditoría del 3 de octubre confirmó en la instantánea 2 de AdventureWorks 6 esquemas, 71 tablas, 482 columnas y 90 relaciones; en la instantánea 3 de WideWorldImporters, 4 esquemas, 48 tablas, 546 columnas y 98 relaciones. Estos conteos pertenecen a las capturas usadas y no sustituyen la comprobación de una fuente nueva.

8.9. Restauración académica de WideWorldImporters

El script `infra/sqlserver/bootstrap-wideworldimporters.sh` obtiene el respaldo oficial cuando hace falta, restaura la base de forma idempotente y no versiona el archivo binario. En la demostración local agrega el lector a los roles existentes cuyo nombre termina en *Sales*; esto satisface la política de seguridad por fila incluida por WideWorldImporters y permite consultar clientes y pedidos. Esta acción pertenece exclusivamente al aprovisionamiento de la base demostrativa. Una conexión externa conserva los permisos y las políticas definidos por su DBA; la aplicación no eleva privilegios.

8.10. Asistente de datamart supervisado

La pantalla **IA >Asistente de datamart** requiere tres condiciones comprobables: fuente seleccionada, habilitada y probada en sólo lectura; instantánea vigente de esa fuente; y proveedor LLM activo con prueba satisfactoria. La entrada no abre directamente ventas: consulta `GET /api/v1/copilot/catalog` con `connection_id` y muestra los dominios que disponen de perfil, metadatos y validadores. Sólo **Datamart de ventas** está implementado; un dominio futuro puede añadirse al catálogo sin crear otra pantalla.

Al elegir el dominio, el backend devuelve preguntas orientadoras y periodicidades compatibles con la instantánea. React no contiene un modelo dimensional fijo de AdventureWorks. El catálogo se administra

por dominio desde **IA >Catálogo analítico**. Las preguntas guían la interpretación, pero no fijan tablas, columnas ni dimensiones. El analista BI continúa por cinco pasos:

1. escribir el objetivo comercial, seleccionar preguntas orientadoras y escoger una periodicidad soportada;
2. revisar conceptos de negocio propuestos en español y revelar, cuando se necesite, su origen técnico;
3. examinar granularidad, hecho, medidas, dimensiones, uniones, KPIs y plan ETL declarativo;
4. personalizar, de forma opcional, resumen, granularidad, dimensiones, medidas, agregaciones y KPIs ya verificados, con una justificación;
5. aprobar o rechazar con comentario y confirmación explícita de advertencias.

Cada intento se almacena en `app.bi_proposals` y queda ligado a la huella de la instantánea, proveedor, modelo, versión de prompt, entrada, mapa semántico, alcance, validación y decisión histórica del actor. Excluir conceptos, cambiar la necesidad o personalizar decisiones crea otra versión; `source_proposal_id` conserva su origen y nunca sobrescribe el registro anterior. La personalización no llama al LLM y no admite tablas, columnas, relaciones ni fórmulas libres: `POST /api/v1/copilot/proposals/{id}/revisions` aplica únicamente selecciones válidas y repite la validación determinística.

Antes de generar la propuesta, la pantalla combina interpretación de IA y reglas determinísticas para separar los requisitos en cuatro estados. **Directo** identifica una referencia física comprobada; **Derivable automáticamente** identifica una fórmula tipada o una ruta compuesta exclusivamente por FK declaradas. Ninguno exige SQL ni selección manual de tablas. Al generar, FastAPI adjunta la evaluación a la propuesta, aplica las derivaciones inequívocas y construye la matriz necesidad–salida. Si una salida directa o derivable no queda materializada, la validación bloquea la aprobación. **Ambiguo** y **No disponible** requieren revisar o limitar el objetivo; una declaración del modelo no sustituye la evidencia.

La búsqueda de evidencia conserva un subgrafo coherente con toda la necesidad. No combina, por coincidencia nominal, medidas de compras con dimensiones de ventas. En la instantánea 2 se contrastaron 23 columnas críticas y siete FK contra SQL Server: fecha, cliente, persona o tienda, territorio, producto, categoría, cantidad, precio, descuento, importe y costo coincidieron en nombre, tipo y nulabilidad. La viabilidad de ventas parte de `Sales.SalesOrderDetail`; el costo llega a `Production.Product.StandardCost` mediante la relación compuesta declarada, sin depender de nombres de tabla fijos.

En cualquier fuente se excluyen del grafo de negocio las relaciones de auditoría. Esto incluye campos como `CreatedBy`, `ModifiedBy`, `UpdatedBy` y `LastEditedBy`. Una FK válida físicamente no demuestra que la entidad relacionada sea producto, cliente o territorio. Las tablas de compras, archivo, pruebas o preparación reciben penalización cuando existe evidencia comercial actual. Las dimensiones directas o derivables requeridas por la necesidad se incorporan desde esa evidencia aunque el proveedor las omita; si no pueden demostrarse, la aprobación se bloquea.

La sección **Versiones generadas** consulta el servidor por dominio y estado, muestra inicialmente **Lista para revisar** y pagina los resultados. Permite abrir cualquier artefacto persistido y escoger cuál revisar o aprobar sin volver a consumir el proveedor. Esto facilita preparar una ejecución local lenta antes de una demostración y conservar, al mismo tiempo, resultados cloud comparables. Los permisos `copilot.proposals.read`, `copilot.proposals.generate` y `copilot.proposals.review` separan consulta, generación y decisión.

Cuando se abre una versión histórica, el asistente restaura el objetivo, las preguntas y la periodicidad persistida; no reutiliza el valor predeterminado del dominio. Las cinco etapas quedan disponibles como consulta inmutable y la acción **Crear nueva propuesta** inicia un borrador vacío en el mismo dominio

sin cerrar sesión ni modificar el expediente consultado. Esta salida elimina la dependencia operativa de reiniciar la sesión para continuar el trabajo.

En **Personalización**, cada medida separa su etiqueta de negocio del identificador técnico y muestra su origen calificado y su regla ejecutable. Por ejemplo, el conteo de transacciones expone `Sales.SalesOrderDetail.SalesOrderID` y `COUNT(DISTINCT SalesOrderID)`; una medida calculada muestra primero la expresión por fila y después la agregación. KPI, selectores y botones conservan el texto completo mediante una disposición adaptativa, evitando concatenaciones como `importe_brutoImporte de ventas`.

8.11. Sugerir y comprobar una necesidad desde los metadatos

El paso Necesidad ofrece dos entradas: redactar un objetivo propio o usar **Sugerir necesidades con esta fuente**, disponible incluso con el texto vacío. **Ayúdame a formular la necesidad** mejora un objetivo existente. Ambas acciones reciben la estructura real, no sólo la huella de una instantánea. Cada sugerencia presenta su propósito, requisitos, referencias comprobadas y límites; **Usar esta necesidad** es una decisión explícita y no modifica el catálogo ni crea un datamart.

Antes del envío se debe revisar y marcar el consentimiento asociado al proveedor y modelo mostrados. El paquete incluye necesidad, nombres de esquemas, tablas y columnas, tipos, claves y relaciones; excluye filas, muestras, propiedades de conexión y credenciales. El servidor verifica una huella del destino activo. Cambiar configuración, proveedor, URL o modelo invalida la autorización anterior; restaurar un borrador tampoco restaura el consentimiento. Una estructura que excede el límite de contexto se detiene con una explicación en lugar de enviarse truncada silenciosamente.

La revisión de viabilidad conserva exactamente el objetivo escrito. El asesor `need_advisor` comprueba referencias, tipos compatibles y rutas FK dirigidas; no invierte relaciones que multipliquen el grano ni acepta costos desconectados. Para una periodicidad exige una fecha comercial verificable y prioriza esa fecha frente a una marca de modificación. Si la solicitud pide facturas, no basta con encontrar pedidos ni una factura aislada: debe existir evidencia relacionada del evento solicitado.

La migración aditiva `20261003_19` crea `app.business_need_reviews`. Conserva la evaluación, proveedor/modelo, actor, instantánea y huellas de entrada y resultado. La generación exige una revisión persistida coincidente; cambiar objetivo, preguntas, período o fuente obliga a repetirla y las respuestas tardías se descartan. Estos controles no reescriben propuestas aprobadas ni ejecuciones existentes.

Límite operativo: respaldo estructural no acredita calidad de filas, suficiencia histórica ni significado financiero. Un costo último registrado no equivale al costo histórico de cada venta. El analista todavía revisa el contrato y aprueba; el ETL todavía concilia. Si una propuesta omite un margen solicitado, disponer de sus operandos no basta: debe contener el resultado derivado calculable o quedar bloqueada.

La prueba autorizada con Claude Haiku mostró por qué estos controles siguen siendo necesarios. Algunas sugerencias de WideWorldImporters y una reformulación se rechazaron por ancla, rutas o tipos; un control local con la misma instantánea sí demostró sus referencias, por lo que no se modificaron los metadatos para aceptar la respuesta IA. La recuperación permite un solo reintento con causas de validación concretas, el mismo destino consentido y las mismas reglas. Si falla de nuevo, el analista recibe el límite.

En AdventureWorks, una revisión llamó costo a `UnitPrice`. La prueba falló desde el criterio BI y motivó un control común de roles en español e inglés: precio no es costo, tasa no es importe e identificador no es medida monetaria. La revisión `need-review-2` impide reutilizar una evaluación anterior como si hubiera superado el control nuevo. Las referencias existentes no validan por sí solas una afirmación libre sobre impuestos o estados; el texto IA continúa marcado para revisión.

En la repetición de WideWorldImporters se obtuvo una sugerencia utilizable y otra bloqueada. La revisión final mostró seis requisitos directos, dos automáticos y cuatro advertencias, sin requisitos no disponibles. Aceptar explícitamente los cuatro límites habilitó la generación, pero en este ensayo no se generó otra propuesta ni otro ETL. El caso negativo de clima externo identificó información ausente y no confundió sensores de refrigeración con datos climáticos. El expediente QA-NEED conserva tanto estos resultados como los intentos fallidos; no garantiza toda salida futura del LLM.

En el cierre de AdventureWorks, la reformulación continuó no utilizable después de dos intentos porque incluyó una capacidad sin validador; no se adoptó. Conservar el texto del analista sí permitió revisar la comparación mensual de importe registrado y unidades por producto, con trazabilidad al pedido. La viabilidad final mostró once requisitos directos, uno derivable, cinco advertencias o ambigüedades y ninguno no disponible. Aceptar los cinco límites habilitó **Generar propuesta**, sin ejecutar esa acción ni crear un ETL. El analista puede continuar con su propio texto; no debe forzar la adopción de una sugerencia rechazada.

Los requisitos interpretados por IA se identifican como **Referencia estructural comprobada**, **Derivación candidata** e **Interpretación propuesta por IA (no ejecutable)**, según corresponda. Una fórmula explicativa del proveedor no se rotula como fórmula controlada: sólo el contrato validado posterior puede alimentar una transformación ejecutable.

8.12. Corrección posterior al cierre: revisión del 4–5 de noviembre de 2026

La preparación de la sustentación reprodujo defectos del motor general, sin abrir un sprint nuevo. El corte vigente usa `need-review-6`, `sales-bi-v7` y `compilation_rules_version=3`. Las pruebas operativas previas conservan sus versiones: AW 122/18 se generó con revisión 4 y WWI 124/19 con revisión 5; ambas propuestas almacenan reglas 2. No se reetiquetan como ensayos del motor posterior.

Intención y evidencia estructural. La trazabilidad genérica de línea a pedido usa la capacidad existente `transactions`, `kind=identifier` y `identity_references` verificadas. No solicita por sí sola una factura. `invoice_event` exige intención positiva en objetivo y preguntas originales, respetando exclusiones, y prueba estructural: el propio hecho puede ser cabecera de factura con PK completa declarada y referencias exactas, o existir una FK completa hacia esa cabecera. Un nombre o una PK parcial no bastan. Una contradicción técnica persistente tras una reparación produce HTTP 422 sin una revisión continuable por aceptar advertencias.

El ranking de agregados es una operación existente, no una columna física ni una capacidad ejecutable nueva. La revisión comprueba primero medida numérica, dimensión y ruta compatible con el grano; luego enlaza `analytical_operation`, aunque los requisitos aparezcan en otro orden. La cobertura exige ambas entradas. Las tildes y la puntuación no cambian esa clasificación; excluir percentiles no excluye un ranking ordinario de sumas. No se añaden percentiles ni predicción.

El escáner determinístico distingue peticiones positivas de cautelas coordinadas ES/EN. «Sin asumir moneda, utilidad ni composición fiscal» no solicita margen; «no excluir» conserva intención positiva. Una pregunta positiva no desaparece porque otra cláusula la niegue. Se documenta una corrección léxica acotada, sin prometer comprensión exhaustiva de negaciones naturales.

Recetas y población. PK, FK e identificadores no son operandos monetarios. El promedio por documento se compila como suma del importe dividida entre conteo de documentos distintos, con ambas medidas verificadas. No se divide un importe entre el valor numérico de un ID ni se sustituye un conteo de líneas por pedidos conservando su rótulo. El contrato transaccional excluye explícitamente el conteo

de líneas que no modela. Reglas 3 restringe la señal nominal `Number`/`Numero` a semántica de entidad o documento y conserva las claves reales: `NumberOfUnits` y `NumeroDeUnidades`, sin PK/FK, pueden ser cantidades SUM.

Versiones, reproducción y aprobación. La expansión y validación reproducen reglas 1 cuando falta marcador, reglas 2 cuando el documento lo declara y reglas 3 para contratos nuevos. No se migran JSON, datos, aprobaciones o huellas históricas. Una aprobación nueva de borrador con prompt o reglas anteriores recibe HTTP 409 y requiere regenerar o personalizar una hija vigente con `source_proposal_id`. Un aprobado legado solicitado idempotentemente retorna intacto.

La comprobación `proposal.replay.passed` informa igualdad real de hashes; la diferencia entre versiones es una advertencia separada. El detalle muestra prompt y reglas persistidos y actuales; sin decisiones guardadas lo declara. La auditoría final de sólo lectura de 91/98/117/120/122/124 preservó documento, validación y `approval_safe` verdadero en las seis. Cinco replay coinciden; 91 conserva su discrepancia preexistente, sin ocultarla. 122/124 siguen reglas 2. No existe un campo histórico de revisión de necesidad en esos documentos: las revisiones 4/5 se atribuyen por el corte operativo observado, no por una columna inventada.

Editor efectivo y continuidad. El editor enlaza medidas y KPI por nombre/código con el contrato compilado, no por índice ni referencia cruda. Muestra origen, función, agregación y fórmula efectivos; las exclusiones aparecen desmarcadas con motivo. La razón conserva sus dos entradas y retirar el denominador retira su KPI. El envío contiene sólo selecciones editables; no modifica el documento original ni su huella. Después de guardar se adopta la hija y se ignoran recuperaciones/evidencias tardías del padre. Desde una aprobada o invalidada, **Crear versión corregida desde ésta** habilita sólo el borrador de hija del paso 4. Cancelar restituye la consulta y los valores originales. Guardar crea y selecciona hija, sin aprobarla ni mutar el padre. La prueba UI de 124 habilitó justificación/guardado y canceló sin crear otra hija.

Períodos y alcance interpretativo. El lector agrupa por periodicidad efectiva del KPI o herencia de propuesta. Normaliza los códigos de catálogo `day`, `week`, `month`, `quarter`, `year` y los códigos KPI `daily`, `weekly`, `monthly`, `quarterly`, `yearly`. Las semanas son ISO y los subtítulos/etiquetas reflejan el período. Una matriz de 22 consultas reales sobre 16/17, con configuración copiada en memoria y transacción de sólo lectura, comprobó dos configuraciones persistidas, diez códigos KPI y diez herencias. No fueron 22 ETL. Las sumas conciliaron con diferencia cero salvo residuo AW trimestral de $1{,}49 \times 10^{-8}$, menor que tolerancia 10^{-6}.

El proveedor del copiloto devuelve sólo respuesta y evidencia. La plataforma construye hasta tres sugerencias desde métricas agrupables y dimensiones materializadas y usa el mismo catálogo para interpretar intención. La cautela pública es determinística, íntegra y visible primero entre los hallazgos ejecutivos. Usa procedencia de contrato/instantánea, fuentes, columnas, claves y la misma ruta FK del materializador; los nombres derivados conservan operación y linaje sin transmitir valores personales. Un origen directo se distingue de una ruta FK. La ventana real del ETL no es lectura exacta, última modificación del origen, prueba de vigencia a fecha del hecho ni entrega verificada. Un ranking tampoco prueba preferencias, demanda, eficacia de campañas, mercado o causalidad.

Recorridos reales conservando la necesidad. AW mantuvo el objetivo detallado de 532 caracteres que había fallado en 118. La nueva 121 y su hija 122 aprobaron cuatro KPI, tres medidas, cuatro dimensiones, claves de detalle/pedido, seis referencias, cero errores y nueve avisos. La huella

`28ac4d6997d8` coincide con reconstrucción. El ETL 18 completó 26 operaciones, cinco tablas y 121.317 filas por lado, diferencia cero; importe 109.846.381,399888 USD, 274.914 unidades, 31.465 pedidos y razón 3.491,065672966407. Se validaron tres equivalencias de grupos y seis códigos excluidos, preservando originales. En 2013/Northwest se comprobaron 6.015.174,735836 USD, 17.452 unidades y 2.053 pedidos. El chat final conservó Top 3 y denominador, evitó inferencias de preferencias/causalidad y presentó tres sugerencias disponibles.

WWI repitió los mismos 693 caracteres, cuatro preguntas y mes. Bajo revisión 5 no apareció el falso margen ni su ACK. La 123 y su hija 124 aprobaron cuatro KPI, siete referencias, cero errores y siete avisos, huella `b5872c3bbd6d` igual a reconstrucción. El ETL 19 completó 26 operaciones, cinco tablas y 228.265 filas por lado, diferencia cero; importe de origen 198.043.439,45, 8.950.628 unidades, 70.510 facturas y razón 2.808,73. La fuente no acreditó moneda única. En 2015/California: 1.215.705,55, 57.142 unidades, 481 facturas y razón 2.527,454365904366. El chat final repitió Top 3 54.648 / 50.715 / 45.860,85 sobre el total territorial, distinguió ficha extraída durante ETL de historia/entrega no probadas y el clic real de sugerencia devolvió cinco clientes con denominador 57.142. La sospecha previa sobre nombres se descartó contrastando 663/663 etiquetas con `CustomerName`, sin diferencias ni cambios por apariencia del nombre.

Los ejemplos previos 117/16 y básico 120/17 se conservan como evidencia anterior; el básico de dos KPI no sustituye el objetivo detallado. El intento 118 y la primera interpretación temporal errónea del chat 19 permanecen en QA histórica.

Corte de verificación y demostración. `make verify` V4 terminó con código 0 el 5 de noviembre: 454 pruebas backend, un aviso Starlette/AnyIO sin fallo, 46 frontend, Ruff/formato en 91 archivos y Mypy en 57, más controles de rama, Compose, workflow, lint, construcción y documentos. Las 266 focales de necesidad/compilación son subconjuntos (156 y 110); también lo son las 53 analíticas y 52 de exportación. No se suman al total global. El primer fallo integral por nombre perdido en preset LLM se conserva como incidente resuelto; no se presenta ese intento como aprobado. Correo de edición es de sólo lectura, cuentas protegidas omiten roles y errores/carga de catálogos bloquean el guardado. Persiste el límite de 100 elementos de esos selectores. No se probaron sesiones de todos los roles ni se cambiaron cuentas para ello.

La demostración puede consultar aprobados, expedientes conciliados y reportes sin nueva llamada LLM ni nuevo ETL; una conversación registrada no es una llamada en vivo. Generar una necesidad nueva requiere disponibilidad, permisos y consentimiento; no se garantiza determinismo del proveedor ante el jurado. Fuentes, estado de cada incidencia y métodos de QA constan en `docs/testing/qa-postcierre-2026-11-04.md`. El código `bf50172` se publicó en la rama correctiva y el PR 57 hacia `develop` se abrió como borrador; no se declara fusionado ni promovido a `main`.

Ajuste posterior de unidad y legibilidad en conciliación. La inspección adicional del 5 de noviembre reprodujo en el expediente 18 una etiqueta de promedio que invadía el valor y una razón rotulada como moneda de origen por transacción pese a USD verificado. La corrección del frontend permite contraer las dos columnas, envolver etiquetas y valores y apilarlos hasta 760 px. El formato interpreta la unidad completa: sustituye el prefijo monetario genérico por la divisa comprobada y conserva el denominador por, per o barra. ISO de tres letras y razones conservan dos decimales, al igual que cantidades fraccionarias o unidades desconocidas; sólo los conteos del conjunto cerrado usan enteros. Estos alias no dependen del nombre ni código del KPI. Se mantiene únicamente la compatibilidad previa de dos códigos de catálogo que omitían por unidad.

El ajuste afecta `frontend/src/App.tsx`, sus pruebas y `frontend/src/styles.css`; no cambia contrato,

fórmulas, valores, aprobaciones, backend ni datos. La nueva inspección consulta 18/19 conservados, sin otro ETL ni llamada LLM. Las capturas y exportaciones previas conservan su aspecto e identidad: no reciben retrospectivamente el resultado de la UI corregida. El corte V4 de `bf50172` (454/46) precede al ajuste `8f1c7d1`. `make verify` V6 cerró con salida 0, 454 backend y 48 frontend. V5 conserva su interrupción 143 y no se presenta como cierre aprobado. La lectura real de 18/19 a 574 y 1366 px no mostró solapes: AW presentó USD 3.491,07 por transacción y WWI conservó 2.808,73 moneda de origen por transacción. Las nuevas capturas 126/127 tienen evidencia propia; la regeneración y revisión documental se registran aparte.

8.13. Validación acumulativa visible

La versión seleccionada incorpora **Validación estructural de la propuesta - Sprint 3**. La acción **Verificar evidencia** llama a `POST /api/v1/copilot/proposals/{id}/verify`; FastAPI no vuelve a consumir el LLM, no ejecuta el ETL y no modifica el estado de la propuesta. Comprueba la huella de los metadatos, referencias técnicas, consistencia del contrato BI y reconstrucción determinística desde `ai_decisions`. Una diferencia entre la propuesta persistida y la reconstruida se presenta como advertencia de compatibilidad, no como prueba de corrupción. La seguridad vigente depende de la integridad de la instantánea, las referencias comprobadas y la ausencia de errores estructurales.

Este control permite al analista BI verificar en Sprint 3 que la estructura utilizada procede del OLTP y que la propuesta aprobada no cambió. Sprint 4 agrega un expediente de ejecución independiente que compara filas al grano aprobado, pedidos distintos, unidades, importes y KPI entre AdventureWorks OLTP y el datamart. Sprint 5 agrega cobertura descriptiva, paneles sobre agregados conciliados, chat seguro y reportes fieles. AdventureWorksDW permanece como contraste secundario, no como fuente primaria de verdad. El juicio de expertos pertenece al cierre académico; pronóstico, MAPE y RMSE fueron retirados del alcance por decisión del tutor.

Cada comprobación muestra **Cumple** o **Revisar**; el evento `copilot.proposal.verify` conserva identificador y prefijos de huella sin metadatos completos, filas ni secretos. La reproducibilidad se refiere a reejecutar decisiones persistidas con reglas determinísticas, no a exigir que dos llamadas nuevas al LLM generen exactamente la misma respuesta.

El LLM no constituye la autoridad técnica. Primero interpreta conceptos y luego devuelve decisiones compactas de hecho, medidas, dimensiones y KPI mediante un esquema JSON construido desde el alcance. FastAPI completa mecánicamente PK, FK, atributos, reglas y operaciones ETL; conserva las decisiones originales bajo `ai_decisions` y valida el contrato expandido. También bloquea instrucciones SQL o ejecutables en la entrada y en la respuesta. Una propuesta con errores permanece en **Validación fallida**; sólo una versión válida pasa a **Lista para revisar**.

Cada medida y KPI declara una función tipada: importe, cantidad, conteo de clientes o conteo de transacciones. FastAPI contrasta esa función con columna, agregación y operación, y excluye un KPI que apunte a una medida incompatible. Las frases libres del proveedor se conservan como observaciones de trazabilidad, separadas de ajustes automáticos y advertencias determinísticas. El analista puede excluir o reasignar un KPI a otra medida compatible ya verificada; si no existe, la interfaz explica que debe retirar el KPI o generar una propuesta con la medida faltante. Las dimensiones proceden de la propuesta del LLM y sólo sobreviven cuando sus fuentes pertenecen al alcance verificado; la prueba real identificó `dim_fecha` desde `Sales.SalesOrderHeader.OrderDate`. Sin una fuente comprobable, no se inventa.

La aprobación no crea tablas ni ejecuta transformaciones. Produce un contrato humano y técnicamente aceptado que el módulo ETL puede seleccionar después. La propuesta aprobada compatible más reciente

se preselecciona, pero el analista debe confirmarla y puede comparar otras aprobadas; cada ejecución conserva el identificador elegido. El perfil `ventas` encapsula reglas semánticas, periodicidades soportadas, términos de descubrimiento, destinos y validadores; las preguntas se administran en el catálogo. El registro de conectores desacopla la lectura canónica del motor. Añadir inventario o MySQL exige implementar y probar el perfil o adaptador correspondiente antes de exponerlo.

Una aprobación puede retirarse desde la web con motivo obligatorio y una versión no aprobada puede descartarse. No se eliminan físicamente los registros porque conservan evidencia y auditoría. La migración `20260920_13` amplía la restricción de estados de `app.bi_proposals` con `invalidated` y `discarded`, evitando divergencias entre la lógica de aplicación y PostgreSQL. La verificación es estrictamente de sólo lectura; incluso un error bloqueante vigente impide una carga nueva, pero no revoca por sí mismo una decisión humana. `POST /api/v1/copilot/proposals/{id}/restore-approval` permite restaurar una retirada explícita con motivo y confirmación de advertencias. La migración `20261001_18` corrigió las retiradas automáticas históricas únicamente cuando el mensaje correspondía al efecto lateral anterior y existía un ETL conciliado, dejando un evento de auditoría por propuesta.

8.14. Materialización controlada del Sprint 4

La pantalla **Datos >Datamart de ventas** guía cinco pasos: elegir propuesta, revisar indicadores, confirmar ejecución, materializar y validar. Al entrar, abre primero la ejecución conciliada más reciente de la fuente y muestra la misma pareja ejecución–propuesta publicada en Analítica de ventas. Las propuestas aprobadas no ejecutadas quedan como alternativas para una carga nueva; no sustituyen visualmente al datamart vigente. El catálogo conserva además los contratos con ejecuciones históricas aunque hayan dejado de ser elegibles, pero los presenta en modo de sólo lectura. FastAPI repite las comprobaciones inmediatamente antes de preparar una ejecución nueva.

El analista revisa una cantidad variable de KPI sugeridos por la IA. Cada indicador muestra nombre, propósito, medida, periodicidad, unidad y receta compilada. Las recetas permitidas son agregación, razón y participación; las medidas calculadas aceptan únicamente operaciones aritméticas tipadas y columnas verificadas. No existe un campo de SQL ni una fórmula libre. Si una tasa de descuento se usa como importe, el backend exige precio, tasa y cantidad inequívocos y construye `precio * tasa * cantidad`; si no puede demostrarlo, bloquea la ejecución.

Al confirmar, `POST /api/v1/etl/executions` conserva propuesta, instantánea, selección de KPI, plan, validación, versión del constructor y responsable. La materialización se ejecuta desde FastAPI con la cuenta de origen de sólo lectura y publica cada carga nueva en un esquema controlado propio, `mart_ventas_e{id}`. Dentro de ese esquema la sustitución de tablas es transaccional, pero otra ejecución no se elimina. Las dimensiones reciben claves sustitutas y conservan las claves de negocio; el hecho se carga al final. El esquema legado `mart_ventas` sigue siendo consultable para la última carga anterior al versionado.

El paso **Validar resultados** compara origen y destino y muestra filas, diferencias, tablas, KPI y estado. Para la ejecución 6 leyó y cargó 121317 líneas, con diferencia cero; calculó seis KPI, 274914 unidades y 31465 pedidos distintos. Los importes se muestran con código ISO sólo después de comprobar una moneda única relacionada con el hecho. AdventureWorks confirmó `USD` mediante `Sales.CurrencyRate.FromCurrencyCode`; la acción **Comprobar divisa sin repetir el ETL** enriqueció el expediente existente sin volver a cargar filas. Una fuente mixta o ambigua conserva *moneda de origen* y explica la evidencia faltante.

La prueba multifuente produjo la propuesta 96 y la ejecución 13 sobre WideWorldImporters. El ETL cargó 231412 líneas de pedidos de origen y destino, diferencia cero, 73595 pedidos y 9310904 unidades.

Calculó 177634276,40 de importe de pedidos, 88904313,90 de costo y 88729962,50 de margen según la receta histórica. Las dimensiones publicaron 227 productos, 663 clientes y 53 territorios con etiqueta descriptiva. La auditoría posterior precisó que este expediente no representa facturación emitida y que el último costo registrado no acredita costo histórico por venta. Sus cifras se conservan con esta limitación, no como margen contable certificado. Como la estructura no demostró una moneda ISO única, la interfaz mantuvo *moneda de origen*; como tampoco demostró descuento, lo registró como limitación en vez de inventarlo.

La prueba de cierre alternó ambas fuentes sin cerrar sesión y verificó Inicio, Explorador, catálogo, expedientes, analítica, copiloto y exportaciones. La expresión "territorio líder" se resuelve desde el ranking conciliado vigente antes de construir la agregación: Southwest para AdventureWorks y Texas para WideWorldImporters. Esta resolución no depende del proveedor LLM. El expediente reproducible, con matriz, capturas y resultados, se encuentra en `docs/testing/Informe_pruebas_integrales_multifuente.pdf`.

La interpretación de categorías se ejecuta después de la conciliación numérica. Sólo considera valores descriptivos, no sensibles y de baja cardinalidad; conserva el dato original y propone una columna española separada. La transformación se anuncia como **Revisar en el paso 5**. Allí el analista puede corregir, excluir o publicar cada mapeo y debe confirmar su revisión. La ejecución 6 publicó `group_es` para Europa, Norteamérica y Pacífico, sin traducir códigos de país.

La tabla **Expedientes recientes** permite reabrir cualquier ejecución sin repetir la carga. Si la misma propuesta y selección de KPI ya tiene un expediente preparado, en curso, conciliado o pendiente de revisión, el botón de materialización se reemplaza por **Abrir expediente**; FastAPI aplica el mismo bloqueo dentro de una transacción. Sólo una ejecución fallida permite un reintento controlado.

Una propuesta aprobada conserva **Necesidad**, **Conceptos**, **Propuesta**, **Personalización** y **Revisión** como pestañas de consulta. Del mismo modo, un expediente completado permite recorrer **Elegir propuesta**, **Revisar indicadores**, **Confirmar ejecución**, **Materializar y cargar** y **Validar resultados**. Los campos quedan deshabilitados y la cuarta etapa explica la materialización realizada sin ofrecer una nueva carga. Este recorrido permite sustentar y auditar el proceso completo desde la propia plataforma.

8.15. Copiloto contextual para decisiones semánticas

En el paso de conceptos, la confianza del LLM no equivale a validación. La plataforma agrega evidencia determinística de tabla, columnas, clave y relaciones. Los conceptos de confianza baja o sin respaldo suficiente nacen desmarcados. **Ver evidencia y cómo resolver** explica causa, riesgo y consecuencias; el analista no necesita abrir DBeaver ni escribir SQL.

Cada concepto puede abrir un diálogo contextual. La IA recibe el objetivo, las preguntas y la evidencia estructural permitida; nunca recibe filas, credenciales o SQL libre. Su respuesta separa recomendación, confianza, evidencia citada, riesgo y acción. **Aplicar recomendación** sólo cambia la casilla visible y deja la decisión final al analista. La migración `20260923_15` conserva estas conversaciones con proveedor, modelo, responsable y fecha.

8.16. Resolución descriptiva y ejecución corregida del Sprint 5

La conciliación numérica no basta para publicar una dimensión. La ejecución 6 conservó correctamente la población de `Sales.Customer`, pero no recorrió las relaciones opcionales hacia personas y organizaciones; por ello `dim_cliente` mostraba números de cuenta. No era correcto reemplazar la población por todas las filas de `Person.Person`, porque allí también existen empleados y otras entidades que no compraron.

El resolutor del Sprint 5 separa **población** de **identidad**. Parte de la entidad vinculada al hecho y recorre claves foráneas verificadas hacia atributos descriptivos. Clasifica ramas como persona, organización o entidad relacionada; combina nombres mediante operaciones tipadas, conserva un respaldo explícito y bloquea relaciones que multipliquen el grano. La decisión no depende de que una tabla se llame *Customer* o *Person*: usa función semántica, relaciones, tipos, cobertura y atributos.

Antes de aprobar una propuesta, la interfaz permite **Revisar identidad de dimensiones**. Presenta cobertura, número de entidades, registros sin nombre, origen, columna destino y una muestra limitada de clave, tipo y etiqueta. La muestra permanece dentro de la plataforma y no se envía al LLM. Una ruta inequívoca se aplica automáticamente; una cobertura insuficiente bloquea la publicación y conduce a una nueva versión supervisada, sin DBeaver ni SQL libre.

La propuesta 54 produjo la ejecución 7. El datamart conserva 121317 líneas, 274914 unidades y 31465 pedidos sin diferencia. `dim_cliente` contiene 19820 filas: 19119 personas y 701 organizaciones; todas disponen de `nombre_cliente` y `tipo_cliente`. Las claves `CustomerID`, `PersonID`, `StoreID` y `AccountNumber` permanecen para linaje y conciliación.

8.17. Analítica explicable y conversación contextual

La opción **Análisis y decisiones >Analítica de ventas** abre únicamente una ejecución exitosa y conciliada. La vista ejecutiva prioriza KPI, evolución, productos, territorios y hallazgos; la vista analítica habilita selección de indicador, clientes principales, tablas de respaldo y calidad. Año y territorio se aplican en el servidor a KPIs, gráficos, hallazgos y exportaciones. La moneda se muestra como `USD` porque fue comprobada en la fuente, no inferida por un símbolo.

Los gráficos admiten exploración directa con ratón, toque o teclado. Seleccionar un punto, barra o categoría abre dentro de la tarjeta un detalle con etiqueta, valor exacto y participación sobre el conjunto visible; cerrar el detalle no altera filtros. En la distribución territorial aparece **Filtrar tablero por...** únicamente cuando la etiqueta coincide con una opción real del catálogo devuelto por FastAPI. Aplicarla actualiza conjuntamente indicadores, visuales y hallazgos; no se aceptan valores escritos libremente.

Las razones `costo_por_unidad` y `venta_por_unidad` no representan una fila o producto individual. Son promedios ponderados sobre el filtro vigente: `CostoTotal / UnidadesVendidas` y `VentasBrutas / UnidadesVendidas`. Por ello la interfaz utiliza **Costo promedio por unidad vendida** y **Venta promedio por unidad vendida**, muestra `USD por unidad` y conserva el valor exacto como ayuda aunque la tarjeta abrevie cifras extensas para evitar desbordamientos.

Los hallazgos determinísticos citan su evidencia y no explican causas. En particular, una diferencia entre el primer y el último mes visible se describe como comparación de extremos; no demuestra tendencia o causalidad y puede involucrar meses con cobertura parcial.

El bloque **Preguntar sobre estos resultados** está separado del panel de hallazgos. Ofrece preguntas guiadas para comprender, decidir e interpretar con cautela en modo ejecutivo, y para comparar, investigar o validar en modo analista. El servidor reconstruye la selección conciliada y envía al LLM sólo KPIs, puntos agregados, hallazgos, calidad, granularidad y filtros. No envía filas, credenciales ni SQL. La respuesta contiene explicación, evidencias, advertencia, preguntas de seguimiento, proveedor y modelo. Cambiar vista o filtros limpia el contexto anterior.

Para una pregunta como "¿Cuáles son los 5 productos más vendidos en Europa?", la primera llamada al LLM sólo completa un esquema JSON cerrado: indicador, dimensión, Top N, orden, año y territorio deben pertenecer a los catálogos enviados por FastAPI. El servidor valida esos valores y construye por código propio una consulta agregada parametrizada; nunca acepta SQL del modelo. La respuesta muestra la interpretación, el total del universo filtrado antes del Top N, la participación de cada elemento

y la procedencia de medida, operación y agrupación.

8.18. Comprobar promedios y evento comercial

Un promedio por documento se calcula como suma del importe dividida por documentos distintos, no como promedio de importes de línea. La auditoría del 3 de octubre contrastó la propuesta 98 y la ejecución 14 de AdventureWorks: 121317 líneas, 31465 pedidos, 109846381,4250 de importe y promedio de 3491,07 por pedido al redondear. Nueve KPI quedaron conciliados. Estas cifras deben leerse con la procedencia y el filtro de ese expediente.

En WideWorldImporters la propuesta 114 y la ejecución 15 usan líneas de factura: 228265 líneas, 70510 facturas, 198043439,45 de `ExtendedPrice` y seis KPI conciliados. El importe incluye impuesto; no debe llamarse venta neta ni cobro. Para 2015, tablero y copiloto coincidieron en 62090220,81, 22250 facturas y 2790,57 por factura. El expediente 13 de pedidos sigue disponible de forma separada. No se asigna una divisa no demostrada.

Antes de aceptar una explicación gerencial, compruebe el evento, numerador, denominador, filtro y limitación. Máximo de una línea no demuestra producto líder; promedio de una línea no demuestra venta media por cliente. Los controles bloquean estas equivalencias y el analista puede excluir KPI incompatibles antes de aprobar. La evidencia detallada se conserva en `docs/testing/Auditoria_operativa_2026-10-03.pdf`.

8.19. Recuperación después de vencer la sesión

Una respuesta HTTP 401 activa el estado global de sesión vencida y conserva la ruta de trabajo. El navegador guarda únicamente el objetivo redactado, códigos de preguntas, periodicidad, paso y el identificador de una versión persistida. Después de iniciar sesión, React vuelve al asistente y obtiene la propuesta mediante `GET /copilot/proposals/{id}` con los permisos renovados. No se guarda el documento completo, la respuesta del LLM, una credencial ni una operación pendiente. El cierre voluntario de sesión elimina también el borrador.

Los mensajes distinguen sesión vencida (401), permiso insuficiente (403) y servicio temporalmente no disponible (503). Los globos de preguntas usan texto blanco sobre fondos `#2166e8` y `#1f63dc`; sus contrastes calculados son 5,08:1 y 5,42:1, superiores al mínimo AA de 4,5:1 para texto normal.

Si una excepción inesperada ocurre durante el renderizado, el límite global de la interfaz muestra **El expediente sigue seguro** y ofrece recargar. La recuperación no ejecuta nuevamente el ETL. Al abrir un expediente desde el historial también se restablece el desplazamiento al inicio del comprobante, evitando que una posición antigua se perciba como una página en blanco.

La ejecución 9, asociada con la propuesta 73, conserva el expediente completo de referencia. Una carga posterior, la ejecución 10 generada desde Anthropic, sustituyó físicamente el esquema legado antes de introducir el versionado y por ello es la única versión antigua que todavía puede consultarse. La analítica no confunde expedientes históricos con datos disponibles: muestra la ejecución 10 como cobertura parcial y omite la 9 del catálogo analítico, aunque ambas continúan disponibles para auditoría. Cuando existe un solo datamart físico se presenta como **Datamart activo** sin un selector redundante y se ofrece acceso a los expedientes; el combo **Datamart analizado** aparece sólo desde dos alternativas realmente consultables. Las cargas creadas con la versión vigente quedan en `mart_ventas_e{id}` y pueden compararse sin repetir ni destruir el ETL anterior.

Al seleccionar un punto, barra o territorio, el detalle mantiene nombre, valor, participación y acciones dentro de la tarjeta. La rejilla adapta sus columnas al ancho disponible y permite partir cadenas extensas;

por tanto, un valor monetario largo o una etiqueta descriptiva no debe superponerse con la tarjeta contigua. Esta regla debe comprobarse en vista ejecutiva, analítica y resolución móvil.

La corrección es neutral al proveedor. Gemini, Groq, Anthropic, Qwen y Ollama pueden redactar nombres distintos, pero FastAPI compila medidas y dependencias a identificadores técnicos. Los KPI con la misma medida, operación y función se consolidan; `AVG(LineTotal)` se muestra como **Importe promedio por línea de venta**, mientras que **Venta promedio por unidad vendida** se calcula como suma de ventas dividida para suma de unidades. Una receta derivada que no resuelve sus entradas se presenta como **No calculable** y nunca recibe confirmación verde.

La ejecución 11 expuso un caso adicional cubierto por HBI-012. El ETL y su conciliación eran correctos, pero una fórmula podía depender directamente de medidas físicas autorizadas sin que existiera un KPI agregado redundante con el mismo nombre. El servicio de analítica construye ahora el conjunto de dependencias, agrega primero las columnas físicas necesarias y resuelve después las recetas derivadas en iteraciones hasta alcanzar un punto fijo. Si no hay avance, el indicador permanece como **No calculable** y cita la dependencia ausente; no se inventa un cero ni se inicia otro ETL.

La validación de la ejecución 11 recuperó margen bruto, margen porcentual, costo promedio por unidad y venta promedio por unidad tanto para el universo completo como para 2013–Europa. Este comportamiento no depende del orden del JSON ni del proveedor que redactó la propuesta, y conserva el filtro, la fórmula y los operandos utilizados en la respuesta.

La prueba integral del 29 de septiembre extendió esta neutralidad a la generación completa. Las propuestas 87–90 de Claude permitieron aislar, en orden, un incumplimiento de esquema, un texto mayor al límite declarado, una entidad vendedora confundida con cliente y una columna relacionada atribuida falsamente al hecho. La plataforma proyecta ahora cada respuesta al esquema común, elimina propiedades excedentes, respeta longitudes y cardinalidades, y realiza como máximo un reintento guiado. Después vuelve a validar el resultado localmente. La selección de identidad puntúa rutas FK, evidencia descriptiva y función compradora, penaliza empleado, vendedor y proveedor, y no depende de nombres específicos de AdventureWorks. Una medida directa sólo se conserva si su columna existe físicamente en el hecho; una medida opcional incompatible y sus KPI se excluyen con diagnóstico, mientras que la cobertura bloquea cualquier requisito obligatorio ausente.

La propuesta 91, generada con Claude Haiku a partir de la misma necesidad, superó los cuatro controles de evidencia con cero errores y cobertura de 19 de 19 requisitos. La dimensión `dim_cliente` se construyó desde la entidad compradora y resolvió dos variantes descriptivas: persona mediante nombre, segundo nombre y apellido, y organización mediante nombre de tienda. La ejecución 12 materializó cinco tablas en `mart_ventas_e12`, cargó 121317 líneas, obtuvo diferencia cero y publicó 12 KPI calculables. `dim_cliente` contiene 19820 clientes, 19820 nombres no vacíos y dos tipos: 19119 personas y 701 organizaciones.

8.20. Exportación PDF y Excel

Los botones **Exportar PDF** y **Exportar Excel** vuelven a construir la selección autorizada en FastAPI. PDF usa A4 horizontal, páginas deliberadas para resumen, tendencia, rankings, hallazgos y calidad; no captura el HTML ni corta tablas entre páginas. Excel conserva valores numéricos, formato de moneda o conteo, un resumen, una hoja por gráfico visible y, en modo analista, una hoja de trazabilidad. La vista ejecutiva no recibe contenido reservado al analista.

Cada descarga registra actor, formato, vista, ejecución, indicador y filtros. Los archivos no contienen macros, vínculos externos, credenciales, filas crudas, SQL ni fórmulas generadas por IA. La evidencia final de la ejecución 12 se revisó página por página y hoja por hoja. El PDF reserva espacio después de

cada salto para impedir que un título quede fuera del margen superior; Excel conserva los porcentajes del dashboard en escala 0–100 y aplica el signo sin multiplicarlos nuevamente. El libro analítico contiene resumen, cuatro hojas con gráfico y datos visibles, y trazabilidad con origen, destino y diferencia.

La precisión de las exportaciones nuevas se decide desde la unidad contractual, sin inferirla del nombre o descripción del KPI. Moneda por unidad, moneda por pedido, razones y cantidades fraccionarias conservan dos decimales; una etiqueta genérica de unidades no demuestra que la cantidad sea entera. Sólo los conteos explícitos admitidos de documentos, clientes o filas se presentan sin decimales. El resumen Excel utiliza `metric.unit`. Ambos formatos imprimen fuente, base y fecha ISO con el offset recibido; una fecha sin zona lo declara. XLSX conserva cadenas literales, sin interpretar fórmulas o hipervínculos automáticos. Las alturas se calculan según ancho real, fuente, palabras y saltos, incluidas celdas fusionadas de contexto. El bloque aprobó 52 pruebas focales y controles de calidad. Los archivos históricos no se reescriben.

Los cuatro artefactos finales 18/19 se comprobaron por hashes propios: diez páginas PDF inspeccionadas y cuatro hojas por XLSX; cifras, razones, sumas, fuente/base, offset y advertencia temporal completa. La hoja muestra la advertencia en 57 pt. La QA XLSX usó ArtifactTool; no se completó Calc sobre estos finales por memoria. La descarga intermedia 19-contexto recortaba visualmente el aviso aunque conservaba el texto; se preserva como incidencia histórica, sin atribuirle el resultado final. Los manifiestos externos identifican originales, SHA-256, controles y método.

8.21. Configuración LLM cloud y local

La pantalla Configuración LLM mantiene nombre, proveedor, URL validada, modelo, estado y resultado de prueba. Elegir un preset conserva el nombre escrito; el formulario se inicializa por identidad del registro y se limpia sólo después de guardar con éxito. Puede existir una sola configuración activa. Para Gemini, Groq, Anthropic o Qwen Cloud se guarda primero el registro y después se usa **Registrar credencial**; cuando ya existe, la acción cambia a **Reemplazar credencial**. La clave se envía una sola vez y nunca vuelve al navegador. La tabla sólo presenta **Credencial pendiente** o **Credencial configurada**.

El identificador del modelo se parametriza porque el catálogo del proveedor cambia. Gemini se probó con `gemini-3.6-flash`. Para baja latencia, Sprint 4 incorporó Groq Cloud con URL restringida `https://api.groq.com/openai/v1` y modelo `openai/gpt-oss-120b`. La pantalla incorpora **Nivel de razonamiento**: automático, mínimo, bajo, medio o alto. Para GPT–OSS en Groq, mínimo se traduce a `low`; bajo, medio y alto se conservan. La prueba reserva salida según el nivel y usa `include_reasoning: false`; la generación aumenta el presupuesto según el contrato. Así se corrigió la clasificación del error 400 que aparecía cuando el razonamiento consumía la salida antes de completar el JSON. Cambiar URL, modelo o razonamiento invalida la prueba anterior.

Anthropic Claude usa exclusivamente `https://api.anthropic.com` y la ruta `/v1/messages`. El preset inicial es `claude-haiku-4-5-20251001` con nivel mínimo, recomendado cuando se dispone de poco saldo por su menor costo y latencia. `claude-sonnet-4-5-20250929` puede escribirse como modelo alternativo si la cuenta todavía lo autoriza y el análisis justifica mayor consumo. FastAPI envía `x-api-key` y `anthropic-version: 2023-06-01`, incorpora el esquema esperado a la instrucción, omite pensamiento extendido y vuelve a validar localmente el JSON antes de usarlo. La clave prestada se registra en la instalación local; no debe copiarse a `.env`, Git, capturas o documentos.

El validador local del contrato es común a todos los proveedores. Groq conserva JSON Schema estricto como primera opción y tres intentos acotados. Ante un rechazo de validación estructurada, el adaptador puede solicitar JSON simple con el mismo esquema incluido en la instrucción. Claude, Gemini y Ollama

pasan por la misma proyección segura y disponen de un único reintento guiado cuando faltan propiedades obligatorias o los tipos son incompatibles. La respuesta se acepta únicamente si supera nuevamente el validador local; de lo contrario se descarta. Esta adaptación no completa datos de negocio, no cambia relaciones, fórmulas, necesidad, alcance ni proveedor.

FastAPI cifra la clave mediante cifrado autenticado y almacena únicamente el texto cifrado en `app.secrets`. La raíz criptográfica se genera automáticamente con permisos restrictivos en `/var/lib/bi-ia-secrets/master.key` y se conserva en el volumen `secret_key_data`, separado de PostgreSQL. Si se borra ese volumen, las credenciales existentes dejan de poder descifrarse y deberán reemplazarse desde la pantalla. No se copia el volumen entre equipos y no se versiona. `GEMINI_API_KEY` y `DASHSCOPE_API_KEY` ya no se utilizan.

Para probar una configuración cloud se registra la credencial, se usa **Probar conexión** y se revisa el aviso verde de éxito o rojo con causa segura. La prueba sale desde FastAPI, descifra sólo en memoria, tiene tiempo limitado y no envía datos del negocio. La auditoría conserva la acción y el proveedor, nunca la clave ni el contenido de la respuesta.

8.22. Selección operativa de Groq o Claude

La prueba de conexión es necesaria, pero no suficiente para seleccionar proveedor. El procedimiento de aceptación debe repetir una necesidad de referencia, generar una propuesta sin aprobarla y comprobar estado, errores, advertencias, cobertura, medidas y KPI. La comparativa del 27 de septiembre produjo los siguientes resultados:

- Groq medio, propuesta 78: conexión correcta y salida estructurada incompleta;
- Groq alto, propuesta 79: conexión correcta y límite temporal después de los reintentos;
- Claude medio y alto, propuestas 80 y 81: contratos completos, pero dos errores bloqueantes de referencia/procedencia y tres advertencias cada uno;
- Groq bajo, propuesta 73: cero errores y advertencias, ejecución 9 totalmente conciliada;
- Claude bajo, propuesta 77: cero errores, siete advertencias y resultado apto sólo para revisión.

La recomendación del ensayo del 27 de septiembre fue **Groq bajo** con **Claude Haiku bajo o mínimo** como respaldo. No constituye una preferencia universal: en la auditoría del 3 de octubre Groq devolvió cuota temporal y Claude completó el caso de facturación después de corregir su adaptador a salida JSON estructurada nativa. FastAPI mantiene la validación común y un reintento acotado por salida agotada; no acepta contratos incompletos ni cambia automáticamente de proveedor. Medio/alto no equivalen a un esfuerzo efectivo distinto en el adaptador Claude sin pensamiento extendido. La selección operativa depende de la prueba real del contrato y de la cuota disponible, no sólo de conectar. La metodología inicial se conserva en `docs/experiments/2026-09-27-groq-vs-claude.md` y sus resultados no se reemplazan.

8.23. Alternativa LLM local con Ollama

Ollama es un respaldo opcional para probar la integración LLM cuando no se dispone de una cuota cloud o se desea evitar usarla. No reemplaza Gemini ni Qwen Cloud, no se inicia con `make up`, no requiere API key y no publica un puerto en el computador anfitrión. Sólo FastAPI, dentro de la red del proyecto, puede acceder al servicio por `http://ollama:11434`.

8.24. Preparar y descargar el modelo

Todos los comandos se ejecutan desde la raíz del repositorio. Antes de la descarga se recomienda comprobar que el equipo tenga al menos 8 GB de disco libre adicionales. El perfil inicia el servicio, pero no descarga automáticamente un modelo:

```
make ollama-up
make ollama-pull
make ollama-status
```

El primer comando activa el perfil `local-llm`; el segundo descarga `qwen2.5:3b`, salvo que `OLLAMA_MODEL` tenga otro valor local aprobado; el tercero muestra los modelos disponibles. Esta opción se adopta como predeterminada porque ofrece respuestas directas en español con menor consumo que `qwen3:4b`. La descarga se conserva en el volumen `ollama_models`. Por tanto, cada programadora debe ejecutar este procedimiento una vez en su propio computador: Git no versiona el modelo, GitHub Container Registry no lo publica y CI/CD no lo descarga.

8.25. Registrar y probar la configuración

En la pantalla **Configuración LLM** crear un registro con los valores siguientes, inicialmente inactivo para no sustituir una configuración cloud que ya esté funcionando:

Campo	Valor
Nombre	Ollama local de prueba
Proveedor	Ollama local
URL de servicio	`http://ollama:11434`
Modelo	`qwen2.5:3b`
Estado	Inactivo hasta que una persona autorizada decida activarlo

Después de guardar, seleccionar **Probar conexión**. FastAPI consulta `/api/tags` sin enviar datos del negocio. El resultado es verde sólo si Ollama responde y el modelo configurado está descargado; si el servicio responde pero falta el modelo, la interfaz muestra una alerta roja comprensible y se corrige ejecutando `make ollama-pull`. `qwen3:4b` sigue disponible para pruebas de mayor capacidad, pero puede requerir más tiempo por su razonamiento interno. La prueba, creación y cambios quedan auditados sin almacenar una clave ni el contenido de una respuesta del modelo.

8.26. Detener sin borrar el modelo

```
make ollama-down
```

Este comando detiene únicamente Ollama y conserva `ollama_models`. También `make down` detiene el servicio si está activo y conserva el volumen. No se debe usar `docker compose down -volumes` si se desea conservar el modelo local y las bases del proyecto.

9. Ejecución desde imágenes de GHCR

Esta modalidad permite levantar una entrega completa sin compilar el código y sin reemplazar desarrollo. Usa los puertos 8080, 18000, 55433 y 51434, además de volúmenes independientes.

9.1. Autenticarse en GHCR

Los paquetes públicos se descargan sin autenticación. Si GitHub indica que algún paquete es privado, la persona debe tener acceso y un token con permiso `read:packages`. El token se introduce por entrada estándar y no se escribe en el comando:

```
echo "$GHCR_TOKEN" | docker login ghcr.io -u USUARIO_GITHUB --password-stdin
```

Precaución. No guardar el token en el repositorio, en `.env` ni en capturas. La variable temporal debe eliminarse o cerrarse con la sesión.

9.2. Configurar la entrega

```
cp .env.release.example .env.release
```

En `.env.release`, cambiar todos los secretos `ChangeMe_*` y confirmar:

```
IMAGE_PREFIX=ghcr.io/lfxas/prototipo-bi-ia
IMAGE_TAG=main
```

Para una entrega académica estable, sustituir `main` por una etiqueta inmutable, por ejemplo `v1.0.0`.

9.3. Descargar e iniciar

```
make release-up
docker compose --env-file .env.release -f compose.release.yaml ps
```

En esta modalidad el frontend usa `http://localhost:8080` y su API aislada usa `http://localhost:18000`. Puede convivir con desarrollo, pero duplica las bases y requiere más memoria. La vista Nginx ligera y la entrega completa no deben usar simultáneamente el mismo puerto 8080.

10. Comprobación técnica

10.1. Comprobación rápida

```
make doctor
```

10.2. Comprobación completa

El cierre del ajuste `8f1c7d1` del 5 de noviembre de 2026 aprobó `make verify` V6 con salida 0: 454 pruebas backend y 48 frontend, controles de rama, Compose, workflow, calidad, construcción y doce documentos. El corte anterior `bf50172` aprobó V4 con 454/46, Ruff/formato en 91 archivos y Mypy en 57. V5 se conserva como intento interrumpido con 143. Los resultados fechados a continuación son antecedentes conservados; el acta QA distingue cada corte.

El corte de regresión del 3 de octubre, posterior al ensayo autorizado del asesor, aprobó `make test`: 231 pruebas backend y 34 frontend en Docker. También aprobaron Ruff, formato, Mypy, ESLint y la compilación estricta TypeScript/Vite. El aviso de deprecación Starlette/AnyIO no produjo fallos. Estos resultados actualizan el corte anterior de 201/33; no convierten los intentos de reformulación bloqueados en respuestas satisfactorias ni sustituyen la inspección visual documental.

```
make verify
```

El control completo valida desarrollo, el perfil de entrega, el perfil local de Ollama y la entrega GHCR; además construye y prueba frontend/backend y regenera los doce PDF oficiales: bitácora, seis informes de sprint, informe multifuente, auditoría QA, guía de formación, Capítulo III y manual técnico. El resultado de cada ejecución completa se registra por separado de los controles focalizados.

10.3. Contrato OpenAPI



Figura 2: Endpoints técnicos disponibles en el Sprint 1.

10.4. Validación de AdventureWorks

La restauración correcta debe producir 71 tablas de usuario. La sonda del contenedor se autentica como `bi_reader`; por tanto, un estado saludable confirma que la cuenta puede conectar y leer.

Para una auditoría explícita, ingresar al contenedor usando variables ya inyectadas y ejecutar consultas controladas. No copiar las contraseñas al historial:

```
docker compose exec sqlserver bash
/opt/mssql-tools18/bin/sqlcmd -S localhost \
  -U "$SQLSERVER_USER" -P "$SQLSERVER_PASSWORD" -C \
  -d "$SQLSERVER_DATABASE" -Q \
  "SELECT COUNT(*) AS tablas FROM sys.tables;"
exit
```

No se debe ejecutar una prueba destructiva sobre datos que deban conservarse. La automatización de inicialización ya verifica las denegaciones de escritura de forma controlada.

11. Operación completa de los contenedores

Todos los comandos de esta sección se ejecutan desde la raíz del repositorio. En este computador:

```
cd "/home/ceo/Proyectos/BI"
```

En otro equipo se sustituye esa ruta por la carpeta donde se clonó el repositorio.

11.1. Diferencia entre pausar, reanudar y retirar

Acción	Comando Compose	Efecto
Pausar	`docker compose stop`	Detiene los procesos, pero conserva contenedores, red y volúmenes.
Reanudar	`docker compose start`	Inicia contenedores existentes sin reconstruir imágenes. Sólo se usa después de `stop`.
Retirar	`docker compose down`	Retira contenedores y red; conserva los volúmenes nombrados y, por tanto, los datos.
Reiniciar limpio	`docker compose down -volumes`	Retira también los volúmenes. Borra las bases del proyecto y obliga a restaurarlas.

Precaución. No añadir `-volumes` para una detención normal. PostgreSQL y AdventureWorks sólo se conservan cuando los volúmenes permanecen.

11.2. Ciclo del ambiente de desarrollo

El ambiente de desarrollo contiene `frontend`, `backend`, `postgres` y `sqlserver`. El perfil opcional `local-llm` puede añadir `ollama`.

Construir y levantar:

```
make up
make ps
```

Pausar y reanudar los mismos contenedores:

```
docker compose stop
docker compose start
```

Retirar los contenedores y la red, conservando las bases:

```
make down
```

Después de `make down`, `docker compose start` ya no aplica porque los contenedores fueron retirados. Para crearlos nuevamente se usa `make up`; los volúmenes existentes se reutilizan.

11.3. Ciclo del LLM local opcional

```
make ollama-up
make ollama-pull
make ollama-status
make ollama-down
```

La secuencia inicia el servicio, descarga el modelo una única vez, lista lo disponible y finalmente detiene sólo Ollama. El último comando conserva el volumen `ollama_models`. El perfil no publica el puerto 11434, no se inicia al levantar desarrollo normal y no forma parte de `compose.release.yaml`; por ello no cambia la superficie de red de una entrega ni agrega una descarga pesada a CI.

11.4. Ciclo de la vista local de entrega

La vista ligera contiene sólo `frontend-delivery`; comparte la API, la red y las bases del desarrollo.

Construir y levantar Nginx en 8080:

```
make delivery-preview-up
docker compose --profile delivery ps
```

Pausar y reanudar únicamente `frontend-delivery`:

```
docker compose --profile delivery stop frontend-delivery
docker compose --profile delivery start frontend-delivery
```

Detener y retirar únicamente ese contenedor:

```
docker compose --profile delivery rm --stop --force frontend-delivery
```

Estas acciones no detienen Vite, FastAPI, PostgreSQL ni SQL Server. Para volver a crear la vista después de retirarla se ejecuta de nuevo `make delivery-preview-up`.

11.5. Ciclo de la entrega completa desde GHCR

Este ambiente usa `compose.release.yaml`, el proyecto aislado `bi-ia-prototype-release` y el archivo `.env.release`.

Descargar y levantar las cinco imágenes:

```
make release-up
docker compose --env-file .env.release \
  -f compose.release.yaml ps
```

Pausar y reanudar sin retirar contenedores:

```
docker compose --env-file .env.release \
  -f compose.release.yaml stop
docker compose --env-file .env.release \
  -f compose.release.yaml start
```

Retirar sólo la entrega completa y conservar sus bases:

```
make release-down
```

Los comandos de entrega no afectan los contenedores ni los volúmenes de desarrollo. La vista Nginx ligera y la entrega completa usan 8080 por defecto; no deben levantarse simultáneamente sin cambiar uno de los puertos.

11.6. Detener todos los ambientes conservando datos

Si están activos desarrollo, la vista Nginx y la entrega completa:

```
docker compose --profile delivery down
make release-down
```

El primer comando retira desarrollo y `frontend-delivery`; el segundo retira el proyecto de entrega completo. Ninguno usa `-volumes`. Si nunca se configuró `.env.release`, se omite `make release-down`.

11.7. Referencia de comandos abreviados

Comando	Función
`make bootstrap`	Prepara secretos locales, construye y espera la salud del primer arranque.
`make up`	Construye cambios y levanta desarrollo en segundo plano.
`make delivery-preview-up`	Añade Nginx en 8080 y conserva Vite en 5173.
`make ollama-up`	Inicia Ollama local opcional sin descargar un modelo.
`make ollama-pull`	Descarga `OLLAMA_MODEL` una vez por equipo.
`make ollama-status`	Lista los modelos presentes en el volumen local.
`make ollama-down`	Detiene Ollama y conserva el modelo.
`make release-up`	Descarga y levanta las cinco imágenes en un proyecto aislado.
`make release-down`	Retira sólo la entrega completa y conserva sus volúmenes.
`make down`	Retira desarrollo y su red, conservando los volúmenes.
`make ps`	Muestra servicios de desarrollo y estado de salud.
`make logs`	Sigue los registros recientes.
`make test`	Ejecuta pruebas en las etapas Docker de backend y frontend.
`make lint`	Ejecuta Ruff, Mypy y ESLint en contenedores.
`make compose-check`	Valida Compose de desarrollo, entrega y perfil Ollama opcional.
`make docs`	Regenera bitácora, informes de Sprint 1 y Sprint 2, y manual.
`make verify`	Ejecuta la verificación completa antes de entregar.

La recarga automática de Vite y FastAPI permite editar código sin instalar sus dependencias en el host.

12. Persistencia y reinicio de datos

12.1. Detener sin borrar

```
docker compose --profile delivery down
make release-down
```

El primer comando cubre desarrollo y la vista Nginx cuando está activa. El segundo cubre la entrega completa aislada. Los volúmenes permanecen y el siguiente arranque reutiliza PostgreSQL y AdventureWorks.

12.2. Reconstruir desde cero

Precaución. El siguiente procedimiento elimina de forma irreversible sólo los volúmenes del proyecto Compose actual. Confirmar que no contienen resultados que deban conservarse.

```
docker compose down --volumes
make bootstrap
```

La restauración de AdventureWorks volverá a descargar el respaldo cuando el volumen esté vacío.

12.3. Qué se versiona

- Se versionan Dockerfiles, Compose, scripts, migraciones, documentación y datos sintéticos aprobados.
- No se versionan volúmenes, contraseñas, tokens, archivos `.bak`, `.dump` ni datos productivos.
- AdventureWorks se reconstruye desde el respaldo oficial y PostgreSQL desde scripts/migraciones.

13. Actualización y reversión

13.1. Entorno construido desde código

```
git pull --ff-only
docker compose up --build -d
make doctor
```

13.2. Entorno basado en GHCR

```
docker compose --env-file .env.release -f compose.release.yaml pull
docker compose --env-file .env.release -f compose.release.yaml up -d
```

Para volver a una versión conocida, fijar `IMAGE_TAG` a una etiqueta anterior y repetir `pull`/`up`. No usar `latest` para entregas evaluables.

Cuando existan migraciones de datos, la reversión deberá seguir el procedimiento de la versión específica. Nunca sustituir una migración por la copia manual de un volumen.

14. Flujo Git y CI/CD

14.1. Desarrollo por rama

La rama predeterminada `develop` integra el trabajo diario y `main` conserva únicamente entregas estables. Las ramas de trabajo parten de `develop` con los prefijos `feature/`, `fix/`, `docs/` o `chore/`.

```
git switch develop
git pull --ff-only origin develop
git switch -c feature/nombre-cambio
# editar y comprobar
make verify
git add .
git commit -m "feat: descripcion breve"
git push -u origin feature/nombre-cambio
gh pr create --base develop --fill
```

El pull request hacia `develop` exige CI aprobada, una revisión y conversaciones resueltas. Cuando la iteración sea estable, se abre un segundo pull request de `develop` hacia `main`; esa promoción vuelve a ejecutar CI y, al fusionarse, activa CD. El procedimiento completo está en `docs/git-workflow.md`.

Las ramas `develop` y `main` tienen protección activa: impiden force-push y eliminación, exigen pull request, una aprobación, conversaciones resueltas y ocho controles de CI con la rama actualizada. La

excepción administrativa se reserva para recuperación y debe justificarse en la bitácora.

14.2. Integración continua

En cada push y pull request hacia `develop` o `main`, CI:

1. valida que los PR de trabajo lleguen a `develop` y que sólo `develop` promueva a `main`;
2. valida el archivo Compose;
3. construye la etapa de prueba del backend;
4. construye la etapa de prueba del frontend;
5. construye las imágenes de infraestructura;
6. compila los documentos LaTeX y verifica que los PDF versionados estén actualizados.

Los trabajos de CI son efímeros: construyen y prueban contenedores en GitHub y los eliminan al terminar. No mantienen un servidor de desarrollo ni modifican los ambientes locales de los colaboradores.

14.3. Entrega continua

Después de un cambio en `main` o una etiqueta `v*`, CD valida y publica:

- `prototipo-bi-ia-backend`;
- `prototipo-bi-ia-frontend`;
- `prototipo-bi-ia-postgres`;
- `prototipo-bi-ia-sqlserver`;
- `prototipo-bi-ia-docs`.

Las ejecuciones de línea base CI 32749237009 y CD 32749237059 finalizaron exitosamente para el commit `ddf2551`.

CD tampoco reemplaza automáticamente un servidor: publica imágenes con etiquetas `main`, `sha-*` y `v*`. Cada computador o proveedor de despliegue selecciona una etiqueta mediante `.env.release`. Así pueden coexistir desarrollo y producción sin compartir red, puertos ni volúmenes.

14.4. Cierre propuesto en Azure

Se recomienda Azure for Students para la demostración académica. La suscripción aporta crédito temporal, no alojamiento gratuito permanente. La arquitectura objetivo usa una VM Ubuntu x64 con 2 vCPU y 8 GiB, Docker Compose, alertas de presupuesto y apagado o desasignación fuera de las pruebas. SQL Server requiere x64 y consume una parte relevante de la memoria; una instancia gratuita mínima no es suficiente para el conjunto completo.

El despliegue seguirá esta secuencia:

1. CI aprueba el pull request y la promoción `develop ->main`;

2. CD publica las cinco imágenes y una etiqueta inmutable;
3. el ambiente GitHub `production` exige la aprobación definida por el equipo;
4. GitHub Actions se autentica en Azure mediante OIDC y permisos limitados;
5. la VM descarga la etiqueta, ejecuta Compose y comprueba salud;
6. se registran SHA, digest, URL, resultado y cualquier reversión.

La suscripción, la VM y el workflow de despliegue permanecen pendientes hasta que las autoras activen Azure for Students y confirmen región, presupuesto y nombres. No se almacenará una contraseña permanente de Azure en GitHub.

Dependabot trabaja contra la rama predeterminada `develop`. Las actualizaciones npm menores y parches se agrupan; los saltos mayores se revisan de forma intencional y separada para evitar mezclar migraciones incompatibles.

14.5. Cómo verificar CI y CD en GitHub

En GitHub se abre la pestaña **Actions**. Cada fila representa una ejecución; el círculo verde significa que todos sus trabajos terminaron correctamente y el rojo identifica una ejecución que necesita revisión. Al abrir una ejecución se puede leer cada trabajo, descargar los artefactos de documentación y conservar el enlace como evidencia para la tesis.

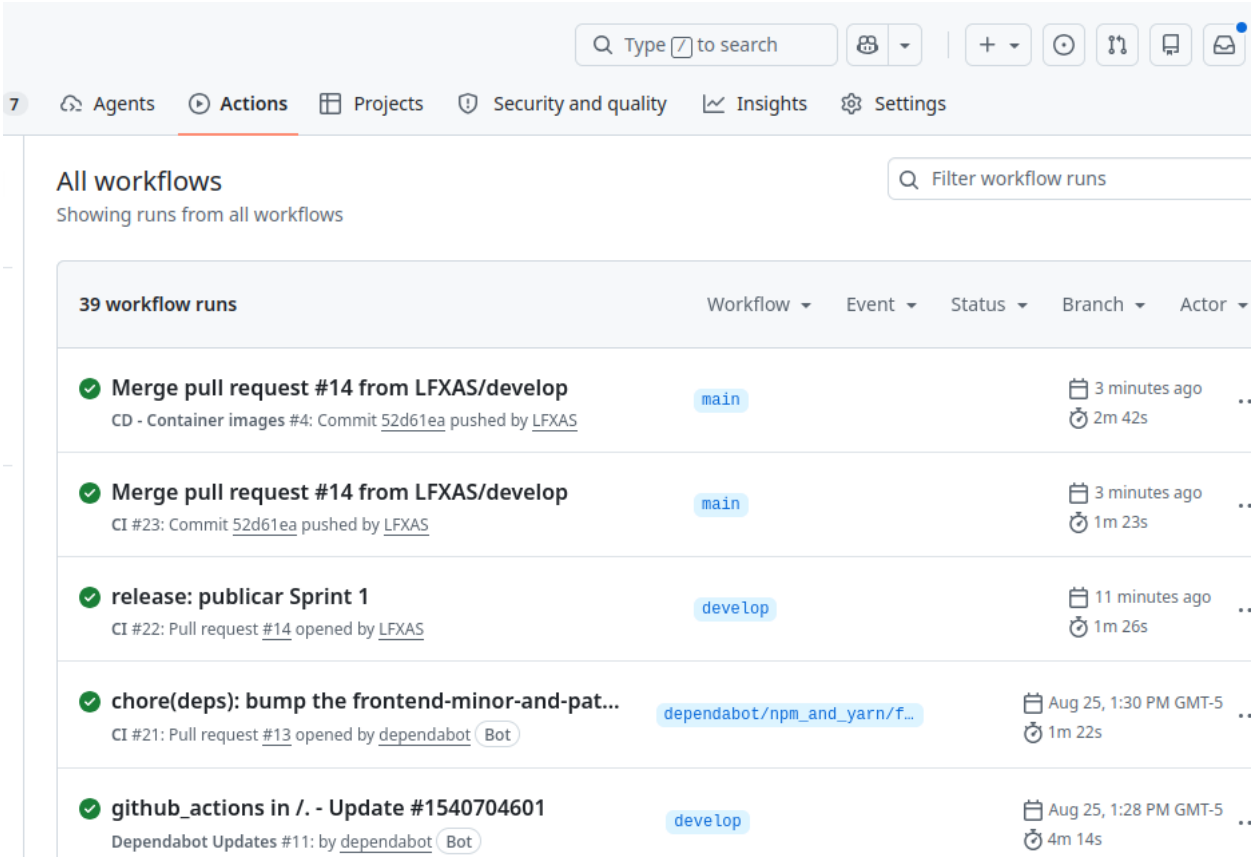


Figura 3: Historial real de GitHub Actions: CI y CD concluyen con estado satisfactorio después de promover `develop` hacia `main`.

Flujo	Cuándo se ejecuta	Resultado esperado
CI	Push o pull request hacia `develop` o `main`.	Valida flujo de ramas, Compose, backend, frontend, infraestructura y los nueve PDF. No publica ni deja contenedores activos ni descarga modelos LLM.
CD	Cambio ya fusionado en `main` o etiqueta `v*`.	Revalida backend/frontend y publica cinco imágenes de producción en GHCR con etiqueta de rama, SHA y, cuando aplique, versión. No despliega aún una VM Azure.
Dependabot	Según su programación.	Abre un PR temporal de actualización; se revisa y prueba igual que un cambio humano.

Para diagnosticar una falla se abre primero el trabajo con marca roja, se identifica el paso que falló y se reproduce localmente con `make verify` desde la raíz del repositorio. No se fusiona un PR fallido ni se intenta corregir modificando directamente `main`. Un cambio documental también pasa por CI porque los PDF versionados deben coincidir con sus fuentes LaTeX.

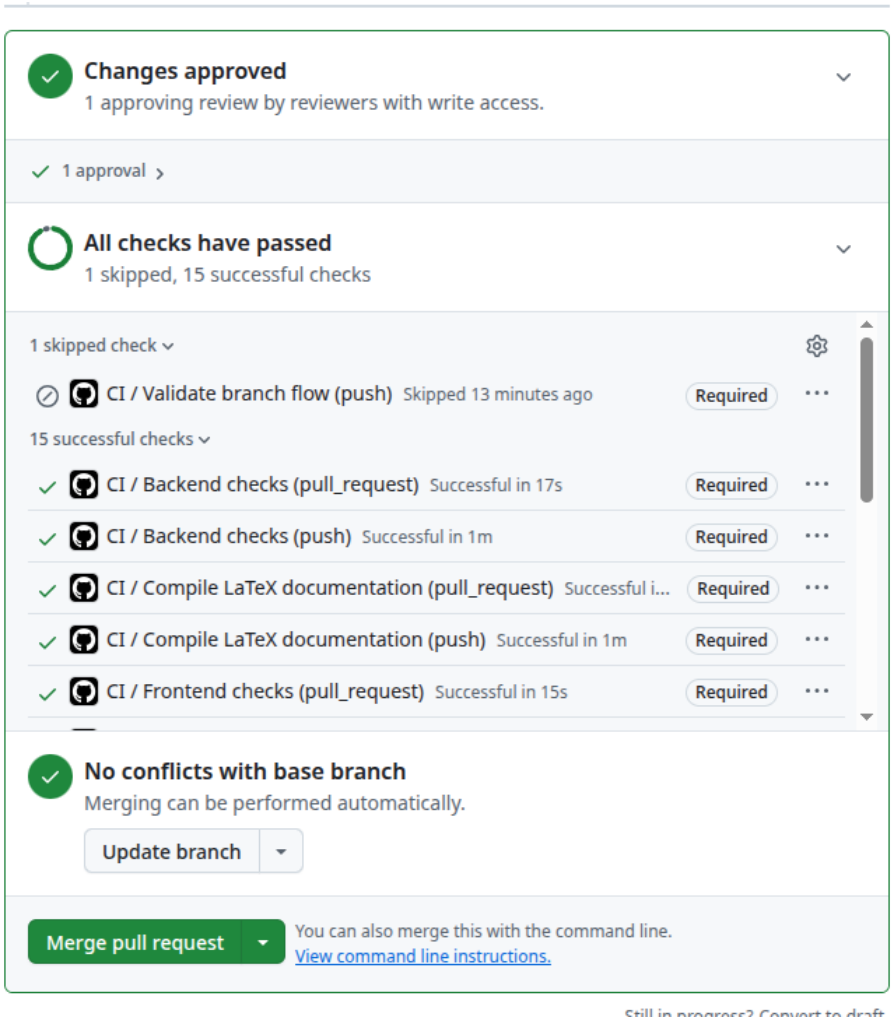


Figura 4: Control de promoción: aprobación humana, verificaciones obligatorias y ausencia de conflictos antes de habilitar la fusión.

14.6. Qué significa DevOps en este prototipo

DevOps no es un contenedor adicional ni una persona distinta. En este proyecto es la práctica de mantener código, infraestructura, pruebas, documentación y entrega dentro del mismo ciclo trazable. Docker evita diferencias entre computadoras; Git conserva el historial; pull requests y protecciones controlan la revisión; CI comprueba cada integración; CD crea imágenes reutilizables; GHCR permite descargarlas; la bitácora y los PDF conservan la evidencia.

El responsable integrador trabaja en una rama de función, corre las verificaciones y abre el PR. Las colaboradoras pueden reproducir el ambiente, revisar, documentar y aprobar. Sólo la promoción aprobada de `develop` a `main` representa una entrega estable. GitHub Actions inicia CD automáticamente después de esa fusión: no hay que presionar un botón de despliegue local y tampoco se modifica una instalación de otra persona.

15. Desarrollo guiado por especificaciones (SDD)

SDD significa *Specification-Driven Development*: antes de programar una capacidad se escribe una especificación breve, verificable y versionada. La intención es evitar que una idea, una propuesta de IA o un cambio de datos se transforme en código sin que el equipo haya acordado su alcance, sus límites y la evidencia que demostrará que funciona.

El Sprint 1 no se rehace con SDD porque ya se cerró como una base técnica. Desde el Sprint 2 la especificación es obligatoria para toda capacidad funcional nueva. Los archivos se conservan en `docs/specs/`; la guía es `docs/sdd-workflow.md` y `docs/specs/TEMPLATE.md` contiene la plantilla.

15.1. Procedimiento SDD para cada función

1. Copiar la plantilla y crear un archivo con el identificador del sprint. Por ejemplo: `SPR-02-rbac-y-parametros.md`.
2. Escribir el problema, objetivo, alcance incluido, exclusiones, actores, datos, API/interfaz, seguridad, criterios de aceptación, pruebas, riesgos y dependencias.
3. Revisar y aprobar la especificación antes de implementar. Si la decisión afecta permanentemente la arquitectura, crear además un ADR en `docs/decisions/`.
4. Crear la rama desde `develop`, implementar dentro del alcance y enlazar la especificación en el PR.
5. Añadir pruebas; actualizar README, arquitectura, réplica, manual y bitácora cuando resulte afectado; ejecutar `make verify`.
6. Con CI verde y revisión aprobada, fusionar hacia `develop`. Registrar el PR, SHA, evidencia y desviaciones en la bitácora.
7. Promover `develop` hacia `main` únicamente cuando el incremento integrado sea estable. CD publicará las imágenes sin sustituir los volúmenes ni los equipos de desarrollo.

> **Nota.** La especificación no sustituye las pruebas. Define qué se debe probar. En especial, cualquier propuesta de IA, SQL o ETL debe declarar aprobación humana, validaciones determinísticas, datos permitidos y auditoría antes de ejecutarse.

16. Construcción operativa del Sprint 1 desde cero

Esta sección explica cómo se concibió la línea base del Sprint 1 y qué debe realizar una persona para reconstruirla manualmente. No sustituye los archivos versionados: los interpreta, relaciona y ordena. Los comandos se ejecutan desde la raíz del repositorio, salvo que se indique otra ruta.

16.1. Secuencia de construcción

1. Revisar el alcance y separar infraestructura de funcionalidades. En el Sprint 1 sólo se autorizaron salud técnica, estructura modular, contenedores, repositorio, automatización y documentación.
2. Crear las carpetas `backend`, `frontend`, `infra`, `scripts`, `docs` y `.github/workflows`. Esta separación permite construir y probar cada componente de manera independiente.
3. Preparar Dockerfiles multietapa. Las etapas `development`, `test` y `production` evitan usar una imagen de desarrollo como entrega estable.
4. Definir `compose.yaml` como orquestador local y `compose.release.yaml` como consumidor de imágenes publicadas. Cada uno usa nombre, puertos, red y volúmenes independientes.
5. Crear `.env.example` y `.env.release.example`; mantener los archivos reales fuera de Git.
6. Implementar sondas de salud y dependencias condicionadas. El frontend no debe declararse listo antes del backend; FastAPI no debe iniciar antes de las bases saludables.
7. Incorporar scripts idempotentes para crear esquemas PostgreSQL y restaurar la fuente SQL Server. Repetir un arranque no debe duplicar objetos ni corromper datos.
8. Centralizar operaciones repetidas en el `Makefile`: levantar, verificar, compilar documentación y ejecutar pruebas.
9. Inicializar Git, crear `develop` y `main`, configurar colaboradores y aplicar protección de ramas.
10. Crear CI para validar cada cambio y CD para publicar sólo aquello que ya llegó a `main` o a una etiqueta `v*`.
11. Ejecutar una prueba limpia, registrar incidentes y conservar PDF, commit, Action y digest como evidencia.

16.2. Primera instalación manual reproducible

```
git clone https://github.com/LFXAS/prototipo-bi-ia.git
cd prototipo-bi-ia
git switch develop
cp .env.example .env
# Editar .env y sustituir cada ChangeMe_*
docker compose config --quiet
docker compose build
docker compose up -d
docker compose ps
curl --fail http://localhost:8000/api/v1/health/live
curl --fail http://localhost:8000/api/v1/health/ready
```

`docker compose config -quiet` detecta variables o estructuras YAML inválidas antes de consumir tiempo construyendo imágenes. `build` procesa los Dockerfiles y crea capas locales. `up -d` crea la red, los volúmenes y los contenedores en segundo plano. `ps` permite comprobar estado y salud. Las dos peticiones HTTP distinguen entre proceso vivo y dependencias realmente disponibles.

17. Referencia comentada de configuración

17.1. Archivos de entorno

`.env.example` es una plantilla versionable para desarrollo. El archivo real `.env` es local. Sus grupos tienen estas funciones:

- `COMPOSE_PROJECT_NAME`: crea un espacio de nombres y evita mezclar redes, volúmenes y contenedores con otros proyectos.
- `FRONTEND_PORT`, `DELIVERY_FRONTEND_PORT` y `BACKEND_PORT`: publican servicios web en el computador anfitrión.
- `LOCAL_POSTGRES_PORT` y `LOCAL_SQLSERVER_PORT`: permiten inspección local; dentro de Docker se siguen usando 5432 y 1433.
- `POSTGRES_*`: identifican la base interna y su cuenta de aplicación.
- `SQLSERVER_*`: identifican la fuente externa, la cuenta lectora, el controlador y las opciones cifradas.
- `VITE_API_BASE_URL`: vacía hace que Vite o Nginx envíe `/api` al backend sin fijar una dirección del host en el código compilado.

`.env.release.example` usa puertos distintos y añade `IMAGE_PREFIX` y `IMAGE_TAG`. El prefijo apunta a GHCR y la etiqueta decide qué entrega se ejecuta. `main` es móvil; una etiqueta como `v0.1.0` es la opción adecuada para una evidencia inmutable.

```
IMAGE_PREFIX=ghcr.io/lfxas/prototipo-bi-ia
IMAGE_TAG=main
```

Los valores `ChangeMe_*` son marcadores, no contraseñas válidas para una entrega. La plantilla puede versionarse porque no concede acceso. Los archivos reales deben aparecer ignorados por Git y comprobarse antes de cada commit con `git status` y, cuando sea necesario, `git diff -cached`.

17.2. Compose de desarrollo: `compose.yaml`

La clave `name` fija el nombre lógico del proyecto. `services` describe los cinco servicios posibles; `volumes` declara persistencia administrada por Docker.

Bloque	Directiva relevante	Razón técnica
PostgreSQL	`build`, `environment`, `volumes`	Construye la imagen propia, inicializa los esquemas y conserva los datos fuera del ciclo de vida del contenedor.
PostgreSQL	`healthcheck` con `pg_isready`	Confirma que el motor acepta conexiones antes de habilitar dependientes.

Bloque	Directiva relevante	Razón técnica
SQL Server	`platform: linux/amd64`	Declara la arquitectura compatible con la imagen de SQL Server.
SQL Server	`ADVENTUREWORKS_BACKUP_URL`	Permite restaurar el respaldo oficial al iniciar un volumen vacío, sin versionar el archivo `.bak`.
SQL Server	archivo de marca y `sqlcmd`	La salud exige restauración terminada y conexión real del usuario lector.
Backend	etapa `development` y montaje `./backend:/app`	Activa recarga automática y refleja cambios del editor sin copiar manualmente archivos.
Backend	`depends_on: condition: service_healthy`	Evita iniciar FastAPI cuando las bases todavía no están listas.
Frontend	montaje del código y volumen `node_modules`	Conserva dependencias Linux dentro de Docker y permite recarga de Vite.
Entrega local	`profiles: [delivery]`	Nginx sólo se crea cuando se solicita; no consume recursos durante desarrollo ordinario.
Todos	`restart: unless-stopped`	Reinicia después de una falla o reinicio del daemon, excepto cuando la persona los detuvo expresamente.

La sintaxis `$VARIABLE:-valor` significa usar la variable del entorno si existe y, de lo contrario, emplear el valor indicado. La forma `$$VARIABLE` escapa el signo para que Compose lo entregue al contenedor y sea el shell interno quien lo interprete.

17.3. Compose de producción local: `compose.prod.yaml`

Este archivo es una sobreescritura para construir las etapas `production` de backend y frontend. Desactiva los montajes de código y la depuración, utiliza Nginx y aplica una política de reinicio más estricta. Se conserva como configuración de construcción; la réplica publicada usa `compose.release.yaml`.

17.4. Compose de entrega: `compose.release.yaml`

La diferencia esencial es `image` en lugar de `build`. El equipo receptor descarga las cinco imágenes ya verificadas desde GHCR. El proyecto predeterminado `bi-ia-prototype-release`, los puertos 8080, 18000, 55433 y 51434 y los volúmenes propios permiten ejecutar desarrollo y entrega a la vez. No se trasladan bases mediante volúmenes: PostgreSQL se inicializa o migra y la fuente de demostración se restaura de forma reproducible.

```
cp .env.release.example .env.release
# Cambiar secretos y seleccionar IMAGE_TAG
docker compose --env-file .env.release \
  -f compose.release.yaml pull
docker compose --env-file .env.release \
  -f compose.release.yaml up -d
```

17.5. Dockerfile del backend

El backend usa una construcción multietapa:

1. `base`: parte de Python slim, instala certificados, ODBC y el controlador Microsoft; copia metadatos y aplicación.
2. `development`: instala dependencias de desarrollo y ejecuta Uvicorn con `-reload`.
3. `test`: copia las pruebas y obliga a pasar Ruff, comprobación de formato, Mypy y Pytest durante la construcción.
4. `production`: instala sólo el paquete, crea un usuario sin privilegios y ejecuta dos trabajadores sin recarga ni depuración.

El patrón hace que una imagen de pruebas no pueda terminar correctamente si falla una comprobación. CI aprovecha esa propiedad construyendo directamente `target: test`.

17.6. Dockerfile del frontend

`dependencies` ejecuta `npm ci` a partir del lockfile, lo que reproduce versiones exactas. `development` inicia Vite escuchando en todas las interfaces del contenedor. `test` ejecuta ESLint, Vitest y la compilación. `build` genera archivos estáticos. `production` parte de Nginx y copia solamente `dist`; por ello no incluye Node.js ni el código fuente de desarrollo durante la entrega.

17.7. Imágenes de infraestructura

- `infra/postgres/Dockerfile`: extiende PostgreSQL y copia scripts SQL a `/docker-entrypoint-initdb.d/`. Esos scripts se ejecutan sólo al inicializar un volumen vacío.
- `infra/sqlserver/Dockerfile`: extiende SQL Server 2022, instala herramientas de descarga y configura un punto de entrada que levanta el motor y ejecuta la restauración idempotente.
- `infra/docs/Dockerfile`: fija Debian, `latexmk` y paquetes TeX. Gracias a esta imagen, el PDF se construye igual en estaciones y GitHub Actions.

18. Automatización local con Make y scripts

Make no reemplaza Docker Compose; proporciona nombres estables para secuencias largas.

Objetivo	Acción y uso
`make env`	Crea `.env` sólo cuando todavía no existe; no sobrescribe secretos locales.
`make bootstrap`	Ejecuta la preparación inicial, el arranque y la espera controlada de todos los servicios.
`make up`/`make down`	Construye y levanta desarrollo o retira contenedores y red conservando volúmenes.

Objetivo	Acción y uso
`make delivery-preview-up`	Construye Nginx y añade sólo la vista de entrega al ambiente de desarrollo.
`make release-up`/`make release-down`	Descarga, inicia o retira el ambiente GHCR usando el archivo de entorno de entrega.
`make test`	Construye las etapas de prueba de frontend y backend; un fallo interrumpe el objetivo.
`make lint`	Ejecuta Ruff, Mypy y ESLint dentro de contenedores desechables.
`make compose-check`	Valida desarrollo, perfil de entrega y entrega GHCR sin iniciar servicios.
`make docs`	Construye la imagen LaTeX y regenera bitácora, informe y manual.
`make verify`	Agrupa configuración, política de ramas, pruebas y documentos; es la puerta local antes del push.

Los scripts de `scripts/` resuelven operaciones que Compose no expresa por sí solo: preparar el archivo de entorno, esperar salud con límite de tiempo, verificar endpoints, validar la dirección de las ramas y probar esa política con casos permitidos y prohibidos.

19. Git y GitHub: procedimiento completo

19.1. Inicialización conceptual del repositorio

En un proyecto nuevo, la secuencia manual equivalente es:

```
git init -b main
git add .
git commit -m "chore: initialize reproducible environment"
git switch -c develop
git remote add origin URL_DEL_REPOSITORIO
git push -u origin main
git push -u origin develop
```

En este repositorio esa inicialización ya se realizó. No debe repetirse dentro de un clon. Una programadora actualiza sus carpetas con `fetch`, `switch` y `pull -ff-only`; no vuelve a clonar cada vez.

19.2. Trabajo diario y revisión

```
git fetch --prune origin
git switch develop
git pull --ff-only origin develop
git switch -c feature/descripcion-breve
# editar archivos
make verify
git status
git diff
git add RUTAS_ESPECIFICAS
git diff --cached
```

```
git commit -m "feat: descripcion verificable"
git push -u origin feature/descripcion-breve
```

En GitHub se abre un PR con base `develop`. Otra persona revisa archivos y evidencia, selecciona *Approve* y la integración sólo se habilita cuando los controles obligatorios están verdes. Para liberar, se abre un segundo PR cuya base es `main` y cuyo origen es `develop`. Esta separación conserva una rama integradora y otra estable.

19.3. Protecciones que deben configurarse en GitHub

Para `develop` y `main` se requiere PR, una aprobación, conversaciones resueltas, controles vigentes, prohibición de force-push y prohibición de borrar. En `main`, la validación adicional rechaza cualquier PR cuyo origen no sea `develop`. Estas reglas son controles preventivos; CI aporta el control detectivo.

20. Integración continua: lectura de `ci.yml`

20.1. Disparadores, concurrencia y permisos

```
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
permissions:
  contents: read
```

CI se ejecuta en pushes y PR relacionados con las ramas permanentes. La agrupación cancela una ejecución anterior de la misma referencia cuando llega un commit nuevo. El permiso `contents: read` aplica privilegio mínimo: CI puede leer el código, pero no modificarlo ni publicar paquetes.

20.2. Trabajos y dependencia entre resultados

- `branch-flow`: sólo en PR; prueba el validador y comprueba base/origen reales.
- `compose`: interpreta las tres configuraciones para detectar errores antes de construir.
- `backend`: construye `target: test`; Ruff, formato, Mypy y Pytest están incorporados en esa etapa.
- `frontend`: construye `target: test`; ejecuta lint, pruebas y compilación.
- `infrastructure`: una matriz construye PostgreSQL, SQL Server y LaTeX en paralelo sin publicar.
- `documentation`: compila los tres PDF, exige que coincidan con los archivos versionados y los adjunta como artefacto de la ejecución.

`actions/checkout` descarga el commit dentro del ejecutor temporal. `setup-buildx` habilita el constructor moderno y caché. `build-push-action` construye sin publicar porque `push: false`. `cache-from` y `cache-to` reducen tiempo, pero no sustituyen pruebas.

El paso `git diff -exit-code` de documentación detecta un error frecuente: modificar LaTeX sin regenerar el PDF. `upload-artifact` permite descargar el conjunto documental desde la ejecución de Actions.

21. Entrega continua: lectura de `cd.yml`

CD se dispara después de un push a `main` o de una etiqueta que comience con `v`. No se dispara directamente desde una rama `feature/*`. La concurrencia no cancela una publicación en marcha, lo que evita dejar una entrega incompleta.

```
permissions:
  contents: read
  packages: write
```

El token efímero `GITHUB_TOKEN` puede leer el repositorio y escribir paquetes, sin almacenar una contraseña personal. El trabajo `verify` valida Compose y vuelve a ejecutar las etapas de prueba. `publish` declara `needs: verify`; por tanto, ninguna imagen se publica si la verificación falla.

La matriz publica cinco componentes. `docker/login-action` inicia sesión en GHCR; `metadata-action` genera etiquetas de rama, versión y SHA; `build-push-action` construye `target: production` y ejecuta `push: true`. El SHA crea trazabilidad incluso cuando la etiqueta `main` avanza.

21.1. Qué hace y qué no hace el CD actual

El CD actual produce y publica artefactos ejecutables. Todavía no crea una VM ni actualiza un servidor Azure. Por ello se denomina entrega continua, no despliegue automático completo. Un despliegue futuro debe seleccionar una etiqueta aprobada, autenticarse mediante OIDC, actualizar el Compose remoto, comprobar salud y conservar el digest.

22. Dependabot y mantenimiento controlado

`dependabot.yml` revisa semanalmente GitHub Actions, Python y npm. El frontend agrupa actualizaciones menores y parches y excluye saltos mayores. Cada propuesta es un PR ordinario: debe pasar CI y ser revisada. Una rama `dependabot/*` es temporal y no representa una rama de arquitectura ni un ambiente.

23. Persistencia, datos y paso a producción

Una imagen contiene software e inicializadores; un volumen contiene estado. No se publica el volumen de PostgreSQL ni el de SQL Server en GitHub. Para una instalación nueva, PostgreSQL crea su estructura mediante scripts y, desde el Sprint 2, migraciones Alembic. Los datos de prueba no deben migrarse a producción. Los catálogos indispensables se cargarán con semillas idempotentes y la configuración variable se creará mediante CRUD y procesos de puesta en marcha.

El esquema `mart` es dinámico: sus estructuras y cargas dependerán del escenario BI aprobado. No debe congelarse un volumen de demostración como si fuera una base productiva. La fuente SQL Server se

conecta con una cuenta de sólo lectura y puede residir en la misma LAN, una VPN o una red alcanzable desde el backend; publicar la aplicación en la nube no le concede acceso automático a una base local.

24. Prueba final y evidencia del Sprint 1

```
make verify
make bootstrap
docker compose ps
make delivery-preview-up
curl --fail http://localhost:8080
```

Se debe conservar: identificador del commit, URL del PR, aprobación, resumen de controles, PDF generado, captura de servicios saludables, pantalla web, OpenAPI, etiquetas y digest de GHCR y resultado de una réplica en otro computador. La captura nunca debe mostrar tokens, contraseñas o el contenido del archivo `.env`.

24.1. Errores de escritura frecuentes

Las opciones de Git no admiten espacios internos. Los comandos correctos son:

```
git fetch --prune origin
git pull --ff-only origin develop
```

Escribir `- prune` hace que Git interprete `prune` como un repositorio. Escribir `-ff - only` produce una ruta extraña. Los caracteres deformados en mensajes en español indican una configuración de codificación de la terminal; no implican que el repositorio esté dañado.

25. Seguridad operativa

1. Cambiar todas las credenciales de ejemplo antes del primer arranque.
2. Mantener `.env` fuera de Git y fuera de los documentos.
3. Usar `bi_reader` para AdventureWorks; no conectar la aplicación como `sa`.
4. Conservar `SQLSERVER_APPLICATION_INTENT=ReadOnly`, pero apoyarse en permisos reales del motor.
5. Gestionar la visibilidad de cada paquete GHCR de forma independiente y no incluir secretos ni datos reales en ninguna imagen.
6. Otorgar acceso únicamente a cuentas identificadas y revocar colaboradores que ya no participen.
7. Revisar los cambios de Dependabot y no fusionarlos sin CI aprobada.
8. RBAC ya exige que FastAPI autorice cada endpoint; ocultar menús en React no es un control de seguridad.

26. Diagnóstico de fallos

Síntoma	Causa probable	Acción recomendada
Puerto ocupado	Otro servicio usa 5173, 8080, 8000, 18000, 55432, 55433, 51433 o 51434.	Cambiar el puerto correspondiente en el archivo de ambiente y reiniciar Compose.
5173 funciona y 8080 no responde	Sólo está iniciado el perfil normal de desarrollo.	Ejecutar `make delivery-preview-up` o iniciar la entrega completa.
Backend vivo, pero `ready` falla	PostgreSQL no está saludable o las credenciales no coinciden.	Revisar `docker compose ps` y los registros de PostgreSQL/backend.
SQL Server no queda saludable	Memoria insuficiente, clave SA inválida o descarga bloqueada.	Asignar 8 GB a Docker, corregir la clave y revisar los registros.
Restauración muy lenta	Primera descarga o emulación ARM.	Esperar y seguir `docker compose logs -f sqlserver`.
SQL Server queda no saludable tras reiniciar	AdventureWorks todavía está recuperándose.	Esperar la sonda; el inicializador comprueba la base antes de recrear la marca de disponibilidad.
Frontend no muestra API disponible	Backend no saludable o proxy mal configurado.	Probar `/health/live`, revisar puertos y registros del frontend.
GHCR devuelve `denied`	Falta autenticación, acceso al paquete o permiso `read:packages`.	Repetir `docker login ghcr.io` con una cuenta autorizada.
PDF no se actualiza	No se regeneró después de editar LaTeX.	Ejecutar `make docs` y revisar los archivos modificados.
Un expediente conciliado muestra KPI como "Sin valor"	Una receta derivada no resolvió todavía sus medidas físicas o agregadas.	Actualizar a la versión con HBI-012 y volver a consultar el expediente; no repetir el ETL. Si persiste, revisar en la trazabilidad la dependencia marcada como ausente.
CI rechaza el cambio	Pruebas, formato, Compose o PDF desactualizado.	Abrir el trabajo fallido, reproducir con `make verify` y corregir antes de fusionar.

26.1. Registros útiles

```
docker compose logs --tail=200 backend
docker compose logs --tail=200 frontend
docker compose logs --tail=200 postgres
docker compose logs --tail=200 sqlserver
docker compose --profile local-llm logs --tail=200 ollama
```

27. Lista de comprobación para una demostración

1. Confirmar que el commit o etiqueta corresponde a la entrega evaluada.
2. Ejecutar `make verify` y conservar la evidencia de CI.

3. Ejecutar `docker compose ps`; los cuatro servicios deben estar saludables.
4. Consultar `/health/live` y `/health/ready`.
5. Abrir frontend y OpenAPI en el navegador.
6. Confirmar que AdventureWorks contiene 71 tablas y el acceso externo usa `bi_reader`.
7. Confirmar que no hay secretos ni archivos `.env` en los cambios de Git.
8. Abrir las ejecuciones 9 y 11 y comprobar 121317 filas conciliadas, 19820 clientes con nombre y tipo, moneda USD y KPI derivados calculables.
9. Probar las vistas ejecutiva y analítica, una pregunta guiada y las exportaciones PDF/Excel con la misma selección.
10. Registrar SHA, etiqueta y digest de imágenes en la bitácora.
11. Conservar los PDF generados para la evidencia de tesis.

28. Mantenimiento de la documentación

Los documentos se compilan dentro de la imagen `bi-ia-docs`; no se instala LaTeX en el host.

```
make logbook
make sprint-report
make sprint2-report
make sprint3-report
make sprint4-report
make sprint5-report
make technical-manual
make docs
```

Después de cambios técnicos se deben actualizar, según corresponda, README, arquitectura, guía de réplica, manual técnico, informe del sprint y bitácora. Los PDF finales se versionan; los archivos auxiliares de LaTeX se ignoran.

29. Responsabilidades y escalamiento

Rol	Responsabilidad
Autoras	Aprobar alcance, secretos, accesos, datos de prueba y entregas académicas.
Colaborador técnico	Trabajar por rama, ejecutar pruebas, actualizar documentación y solicitar revisión.
CI/CD	Repetir controles y publicar imágenes después de la integración autorizada.
FastAPI	Aplica autorización RBAC, validaciones determinísticas, auditoría y pruebas LLM seguras.
Responsable de datos	Confirmar que la fuente externa permanezca en lectura y que no se incorporen datos productivos.

Ante pérdida de datos locales, credenciales expuestas, imágenes inesperadas o permisos de escritura en AdventureWorks, detener la demostración, conservar registros, revocar las credenciales afectadas y documentar la resolución antes de continuar.

A. Referencia rápida de URLs y puertos

Recurso	Dirección	Observación
Frontend desarrollo	`http://localhost:5173`	React/Vite
Frontend entrega	`http://localhost:8080`	Nginx local o GHCR
API desarrollo	`http://localhost:8000`	FastAPI con recarga
API entrega completa	`http://localhost:18000`	FastAPI desde GHCR
OpenAPI	`http://localhost:8000/docs`	Contrato interactivo
Salud básica	`http://localhost:8000/api/v1/health/live`	Proceso de API
Preparación	`http://localhost:8000/api/v1/health/ready`	API y PostgreSQL
PostgreSQL	`localhost:55432`	Acceso técnico local
SQL Server	`localhost:51433`	Acceso técnico local

B. Recuperación mínima

Si el entorno queda en un estado desconocido y no existen datos que conservar:

```
cd "/ruta/del/repositorio"
docker compose down --volumes
docker compose build --no-cache
make bootstrap
make verify
```

Precaución. La recuperación elimina los volúmenes del proyecto actual. No ejecutarla como diagnóstico rutinario ni en una carpeta cuya identidad Compose no haya sido confirmada.

C. Control del manual

Campo	Valor
Estado	Manual acumulativo de los Sprints 1 a 6: construcción supervisada, contexto multifuente, materialización, validación, analítica explicable, conversación contextual y reportería.
Custodia	Repositorio público `LFXAS/prototipo-bi-ia`; escritura restringida a colaboradores.
Audiencia	Autoras, tutor, revisores, colaboradores y lectores académicos.
Actualización	Debe acompañar todo cambio de instalación, puertos, secretos, imágenes, pruebas o recuperación.
Fuente editable	`docs/manual-tecnico/manual-tecnico.tex`.
Compilación	`make technical-manual` o `make docs`.

Historial de cambios

Versión	Fecha	Cambio
1.0.0	24-08-2026	Primera versión: instalación, réplica, operación, CI/CD, seguridad, diagnóstico y recuperación del ambiente del Sprint 1.
1.1.0	24-08-2026	Flujo `feature/*` hacia `develop`, promoción a `main`, protecciones y política de Dependabot.
1.2.0	24-08-2026	Vista Nginx paralela, entrega GHCR aislada y relación entre ramas, CI/CD y ambientes simultáneos.
1.3.0	24-08-2026	Propuesta de cierre DevOps en Azure for Students con VM x64, OIDC, aprobación y control de crédito.
1.4.0	24-08-2026	Ciclo completo para levantar, pausar, reanudar y retirar desarrollo, vista Nginx y entrega GHCR sin borrar volúmenes.
1.5.0	01-09-2026	Explicación ampliada de CI, CD y DevOps; procedimiento SDD, plantillas versionadas y primera especificación del Sprint 2.

Versión	Fecha	Cambio
1.6.0	02-09-2026	Reconstrucción operativa del Sprint 1 y referencia comentada de variables, Compose, Dockerfiles, Makefile, Git, GitHub, CI/CD, persistencia y verificación.
1.7.0	02-09-2026	Evidencias visuales de GitHub Actions y de los controles obligatorios de aprobación, CI y fusión.
1.8.0	11-09-2026	Consolidación del Sprint 2: operación JWT/RBAC, navegación, CRUD protegido, auditoría, parámetros, proveedores LLM y perfil Docker local de Ollama con persistencia y validación segura.
1.8.1	11-09-2026	Corrección de eliminación de usuarios temporales: auditoría preservada con etiqueta histórica de actor y referencia nula mediante `ON DELETE SET NULL`.
1.9.0	18-09-2026	Credenciales LLM administradas desde la web, cifrado autenticado, volumen de raíz criptográfica y migración reproducible como cierre pendiente del Sprint 2.
2.0.0	18-09-2026	Inicio del Sprint 3: conexión SQL Server administrada desde la web, contraseña cifrada, validación de sólo lectura, activación única y cuatro parámetros tipados.
2.1.0	18-09-2026	Introspección determinística, instantáneas JSONB inmutables, hash reproducible, deduplicación y explorador responsive de tablas, columnas y relaciones.
2.2.0	19-09-2026	Asistente supervisado del Sprint 3: solicitud guiada, interpretación semántica, propuesta dimensional versionada, validación determinística, decisión humana, conectores y perfiles de dominio extensibles.
2.2.1	19-09-2026	Selección de resultados persistidos por proveedor/modelo y validación real de Gemini 3.6 Flash con nivel gratuito.
2.2.2	19-09-2026	Nivel de razonamiento por configuración y expediente visible de validación acumulativa con reejecución determinística.
2.2.3	19-09-2026	Contexto Ollama ampliado a 4096 y presupuestos de salida ajustados para impedir respuestas JSON truncadas con `qwen2.5:3b`.
2.3.0	19-09-2026	Asistente de datamart genérico, catálogo dinámico de capacidades, personalización versionada, historial filtrable, validación estructural visible y selección prevista para el ETL.
2.3.1	20-09-2026	Contrato semántico de medidas y KPI, mensajes accionables, reasignación supervisada, retiro de aprobaciones y descarte auditable de versiones.
2.3.2	20-09-2026	Corrección de persistencia para los estados auditables de aprobación retirada y descarte mediante la migración `20260920_13`.
2.3.3	20-09-2026	Catálogo web de necesidades analíticas, validación compatible con versiones históricas, restauración auditada de aprobaciones y exclusión de identificadores como medidas monetarias.
2.3.4	20-09-2026	Incorporación determinística de dimensiones solicitadas omitidas por el LLM cuando existe una fuente inequívoca en los metadatos verificados.

Versión	Fecha	Cambio
2.3.5	20-09-2026	Catálogo analítico separado por permisos y dominio, CRUD de preguntas, múltiples periodicidades y dimensiones propuestas por IA en lugar de predefinidas en la necesidad.
3.0.0	24-09-2026	Operación del Sprint 4: Groq Cloud, selección de propuestas, KPI variables, ETL transaccional, conciliación, semántica española, expedientes recuperables, prevención de duplicados y moneda ISO comprobada.
4.0.0	25-09-2026	Cierre técnico del Sprint 5: identidad descriptiva general, propuesta 54, ejecución 7, panel por perfil, copiloto analítico seguro, PDF/Excel fieles y retiro documentado del pronóstico.
4.0.1	25-09-2026	Endurecimiento integral: diagnóstico Groq, viabilidad y cobertura, relaciones controladas, KPI financieros, agregaciones seguras y recuperación de sesión.
4.0.2	26-09-2026	Propuesta 73 y ejecución 9 conciliada, analítica publicada, pruebas de exportación/copiloto y recuperación explícita ante fallos visuales.
4.0.3	26-09-2026	Promedios por unidad con denominador explícito, tarjetas ejecutivas legibles, contraste del chat y consulta inmutable de las cinco etapas históricas.
4.0.4	26-09-2026	Gráficos interactivos y accesibles, menú profesional por flujo de trabajo e integración segura de Anthropic Claude con Haiku 4.5.
4.0.5	27-09-2026	Datamart único identificado sin combo redundante, enlace a expedientes auditables y detalle interactivo adaptativo sin desbordamientos.
4.0.6	27-09-2026	Comparativa integral Groq–Claude, selección operativa de proveedor y restricción consistente de niveles Claude sin pensamiento extendido.
4.0.7	28-09-2026	HBI-012: resolución neutral de dependencias y KPI derivados de la ejecución 11, con recuperación analítica sin repetir el ETL.
4.0.8	29-09-2026	HBI-013: continuidad desde propuestas históricas, periodicidad fiel y personalización legible con origen y fórmula ejecutable por medida.
4.0.9	29-09-2026	Derivaciones automáticas explicadas, evidencia coherente contra metadatos reales y recuperación JSON de Groq con contrato local común.
4.0.10	29-09-2026	Recuperación universal del contrato, identidad de cliente por función y rutas FK, propuesta Claude 91, ejecución 12 conciliada y corrección visual de PDF/Excel.
5.0.0	01-10-2026	Sprint 6 multifuente: fuentes SQL Server concurrentes, catálogo funcional aislado, evidencia técnica automática, WideWorldImporters, propuesta 96 y ejecución 13 conciliada.
5.0.1	01-10-2026	Verificación histórica de sólo lectura, restauración auditable de las propuestas 86/91 y alineación del expediente publicado entre Datamart y Analítica.

Versión	Fecha	Cambio
5.0.1	01-10-2026	Prueba integral multifuente, resolución determinística de territorio líder, restauración del catálogo legado y verificación visual de PDF/Excel.
5.1.0	03-10-2026	Necesidades sugeridas y revisadas con metadatos, consentimiento por destino, evaluación persistida; auditoría de pedidos/facturas y promedios por documento, con límites de evidencia explícitos.
5.2.0	05-11-2026	Revisión posterior al cierre del 4–5 de noviembre: trazabilidad documental, necesidad positiva de factura, compilación versionada para IDs, razones y conteos, periodicidad efectiva y límites de interpretación del chat. Evidencia previa y regresión posterior separadas.
5.2.1	05-11-2026	Ajuste adicional de presentación: unidad monetaria completa y denominador conservado, precisión de cantidades y conteos, y conciliación sin superposición. Verificación posterior separada del corte V4; originales anteriores intactos.

Registro de revisión

Rol	Nombre y conformidad	Fecha
Autora	_____	_____
Autora	_____	_____
Tutor/revisor	_____	_____

Las firmas formales pertenecen a la tesis entregada y no se versionan si contienen datos personales cuya distribución no fue autorizada.